LoanTap deploys formal credit to salaried individuals and businesses 4 main financial instruments:

Personal Loan

EMI Free Loan

Personal Overdraft

Advance Salary Loan

But the main focus is to interpret the underwriting process behind the Personal Loan only

Given a set of attributes for an Individual, determine if a credit line should be extended to them. If so, what should the repayment terms be in business recommendations?

Additional views We need to track the users previous credit line history and repayment status. Analysing the previous loans tenure and the total liability. As we are focusing more on salaried individual, we need to take salary of the person into consideration.

What is Logistic Regression?

Logistic Regression is a fundamental statistical technique used for solving binary classification problems—where the outcome has only two possible values, such as Yes/No, Approved/Rejected, or Default/No Default.

Instead of directly predicting a class label, Logistic Regression estimates the probability of an event occurring, giving an output between 0 and 1. By applying a threshold (commonly 0.5), we then classify the prediction into one of the two categories.

In simpler terms, Logistic Regression helps answer the question:

"Given a set of inputs, what's the likelihood that a specific event will happen?"

For example, in the context of loan applications, we might use Logistic Regression to estimate the probability that an applicant will repay a loan or default, based on attributes like income, employment history, loan amount, and credit score.

Why Use Logistic Regression?

Logistic Regression is widely used in domains like finance, healthcare, and marketing, particularly when interpretability and decision transparency are crucial. Here's why it remains a go-to method:

1. Interpretability Each coefficient in the model explains the impact of a feature on the outcome, and in what direction. For instance, a positive coefficient for income means that higher income increases the chances of loan repayment.
2. ⚡ Efficiency It's computationally efficient and fast to train—even on large datasets —making it ideal for real-time and scalable systems.

3. Probabilistic Predictions Logistic Regression produces a confidence score (e.g., 85% likelihood of default), which is especially useful for risk assessment, threshold tuning, and making informed business decisions.

4. Handles Mixed Data Types It works well with both numerical variables (e.g., income, DTI) and categorical variables (e.g., home ownership, job title), after basic preprocessing like encoding and scaling.

Why Logistic Regression for Loan Approval?

Loan approval is a classic binary decision problem: either a loan is approved or it's not. Given this nature, Logistic Regression becomes a natural fit for several reasons:

✔ It provides clear, explainable predictions that are crucial for transparency in financial services.

It helps quantify risk by predicting the likelihood of loan repayment or default.

It forms a strong baseline model that can later be improved using more complex techniques if necessary.

It's compliant with the need for regulatory and explainable AI—especially in sensitive domains like lending.

Example: Predicting Loan Approval Using Logistic Regression

Let's say we have a dataset of historical loan applications, with the following features:

```
Annual Income

Credit Score

Loan Amount

Employment Length

Debt-to-Income (DTI) Ratio
```

Our goal is to use Logistic Regression to predict:

```
"What is the probability that a new loan applicant will repay their
loan?"
```

Step-by-Step Workflow

1. Dataset Overview Begin by understanding the dataset using basic commands:

   df.head(), df.info(), df.describe(), df.shape()

2. Data Cleaning & Feature Engineering

```
Identify vulnerabilities such as:
  Missing values
  Duplicates
  Outliers
  Improper data types
  Resolve them using:
  Imputation (mean, median, mode)
  Encoding (for categorical data)
  Outlier treatment (e.g., IQR method, z-score)
  Multicollinearity check (e.g., Variance Inflation Factor - VIF)
```

1. Feature Engineering:

   Create binary flags for key indicators (e.g., bankruptcies, public records)

   Convert employment length, term, and grade into usable numerical formats

2. Exploratory Data Analysis (EDA)

   Univariate Analysis Distribution of individual features

   Bivariate Analysis Relationship between features and target variable

   Multivariate Analysis Interactions among multiple variables

   Use both graphical (histograms, boxplots, countplots, heatmaps) and statistical methods.

3. Data Preparation

   Split the dataset into training and testing sets

   Apply scaling (e.g., StandardScaler or MinMaxScaler) to numerical features

4. Model Building Train the Logistic Regression model:

   from sklearn.linear_model import LogisticRegression model = LogisticRegression() model.fit(X_train, y_train)

Review coefficients to understand which features most impact the prediction.

1. Model Evaluation Use key metrics to assess performance:

Accuracy Overall correctness

Precision Correct positive predictions (minimizing false positives)

Recall Ability to detect actual positives (minimizing false negatives)

F1-Score Balance between precision and recall

ROC AUC Score Overall classification performance

Confusion Matrix Visual summary of classification results

2. Visualizations:

   ROC Curve

   Precision-Recall Curve

 Insights & Recommendations

1. Identify key risk indicators (e.g., high DTI, unverified income, short employment)
2. Recommend approval rules based on high-probability predictors
3. Suggest threshold adjustments based on business goals (e.g., minimize NPAs vs maximize approvals)

Installing Dependencies

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn import metrics
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.metrics import precision_recall_curve
from sklearn.model_selection import train_test_split, KFold,
cross_val_score
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_auc_score, roc_curve, auc,
    ConfusionMatrixDisplay, RocCurveDisplay
)
from statsmodels.stats.outliers_influence import
```

```
variance_inflation_factor
from imblearn.over_sampling import SMOTE
```

Loading Dataset

```
df = pd.read_csv('logistic_regression.csv')
df.head()
```

```
{"type":"dataframe","variable_name":"df"}
```

Dataset Overview

```
df.shape
```

```
(396030, 27)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 396030 entries, 0 to 396029
Data columns (total 27 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   loan_amnt             396030 non-null  float64
 1   term                  396030 non-null  object 
 2   int_rate              396030 non-null  float64
 3   installment           396030 non-null  float64
 4   grade                 396030 non-null  object 
 5   sub_grade             396030 non-null  object 
 6   emp_title             373103 non-null  object 
 7   emp_length            377729 non-null  object 
 8   home_ownership        396030 non-null  object 
 9   annual_inc            396030 non-null  float64
 10  verification_status   396030 non-null  object 
 11  issue_d               396030 non-null  object 
 12  loan_status           396030 non-null  object 
 13  purpose               396030 non-null  object 
 14  title                 394274 non-null  object 
 15  dti                   396030 non-null  float64
 16  earliest_cr_line      396030 non-null  object 
 17  open_acc              396030 non-null  float64
 18  pub_rec               396030 non-null  float64
 19  revol_bal             396030 non-null  float64
 20  revol_util            395754 non-null  float64
 21  total_acc             396030 non-null  float64
 22  initial_list_status   396030 non-null  object 
 23  application_type      396030 non-null  object 
 24  mort_acc              358235 non-null  float64
 25  pub_rec_bankruptcies  395495 non-null  float64
 26  address               396030 non-null  object 
```

```
dtypes: float64(12), object(15)
memory usage: 81.6+ MB
```

```
df.dtypes
```

```
loan_amnt               float64
term                     object
int_rate                float64
installment             float64
grade                    object
sub_grade                object
emp_title                object
emp_length               object
home_ownership           object
annual_inc              float64
verification_status      object
issue_d                  object
loan_status              object
purpose                  object
title                    object
dti                     float64
earliest_cr_line         object
open_acc                float64
pub_rec                 float64
revol_bal               float64
revol_util              float64
total_acc               float64
initial_list_status      object
application_type         object
mort_acc                float64
pub_rec_bankruptcies    float64
address                  object
dtype: object
```

```
df.isna().sum()
```

```
loan_amnt                 0
term                      0
int_rate                  0
installment               0
grade                     0
sub_grade                 0
emp_title             22927
emp_length            18301
home_ownership            0
annual_inc                0
verification_status       0
issue_d                   0
loan_status               0
purpose                   0
```

```
title                   1756
dti                        0
earliest_cr_line           0
open_acc                   0
pub_rec                    0
revol_bal                  0
revol_util               276
total_acc                  0
initial_list_status        0
application_type           0
mort_acc               37795
pub_rec_bankruptcies     535
address                    0
dtype: int64
```

```
# Percentage of null values in each columns
df.isna().sum()/len(df)*100
```

```
loan_amnt               0.000000
term                    0.000000
int_rate                0.000000
installment             0.000000
grade                   0.000000
sub_grade               0.000000
emp_title               5.789208
emp_length              4.621115
home_ownership          0.000000
annual_inc              0.000000
verification_status     0.000000
issue_d                 0.000000
loan_status             0.000000
purpose                 0.000000
title                   0.443401
dti                     0.000000
earliest_cr_line        0.000000
open_acc                0.000000
pub_rec                 0.000000
revol_bal               0.000000
revol_util              0.069692
total_acc               0.000000
initial_list_status     0.000000
application_type        0.000000
mort_acc                9.543469
pub_rec_bankruptcies    0.135091
address                 0.000000
dtype: float64
```

```
numeric_columns = df.select_dtypes(include=['float64', 'int64'])

total_acc_avg = numeric_columns.groupby('total_acc')
```

```
['mort_acc'].mean()

def fill_mort_acc(total_acc, mort_acc):
    if np.isnan(mort_acc):
        return total_acc_avg[total_acc].round()
    else:
        return mort_acc

df['mort_acc'] = df.apply(lambda x: fill_mort_acc(x['total_acc'],
x['mort_acc']), axis=1)

df.isnull().sum()
```

```
loan_amnt                   0
term                        0
int_rate                    0
installment                 0
grade                       0
sub_grade                   0
emp_title               22927
emp_length              18301
home_ownership              0
annual_inc                  0
verification_status         0
issue_d                     0
loan_status                 0
purpose                     0
title                    1756
dti                         0
earliest_cr_line            0
open_acc                    0
pub_rec                     0
revol_bal                   0
revol_util                276
total_acc                   0
initial_list_status         0
application_type            0
mort_acc                    0
pub_rec_bankruptcies      535
address                     0
dtype: int64
```

```
# droping remaining null values

df.dropna(inplace=True)

df.shape
```

```
(370621, 27)
```

```
df.isnull().sum()
```

```
loan_amnt               0
term                    0
int_rate                0
installment             0
grade                   0
sub_grade               0
emp_title               0
emp_length              0
home_ownership          0
annual_inc              0
verification_status     0
issue_d                 0
loan_status             0
purpose                 0
title                   0
dti                     0
earliest_cr_line        0
open_acc                0
pub_rec                 0
revol_bal               0
revol_util              0
total_acc               0
initial_list_status     0
application_type        0
mort_acc                0
pub_rec_bankruptcies    0
address                 0
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"loan_amnt\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 126371.73276411944,\n
\"min\": 500.0,\n        \"max\": 370621.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
14250.184487657203,\n          12000.0,\n          370621.0\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"int_rate\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 131029.5268923581,\n        \"min\":
4.471626037652381,\n        \"max\": 370621.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
13.637437193251328,\n          13.33,\n          370621.0\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"installment\",\n
```

```
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
130861.33635448238,\n        \"min\": 16.08,\n        \"max\":
370621.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
435.2181839399279,\n          379.19,\n          370621.0\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"annual_inc\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3044317.1266362513,\n        \"min\": 4000.0,\n        \"max\":
8706582.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\
n          75187.8719837516,\n          65000.0,\n          370621.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"dti\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 131011.28645335113,\
n        \"min\": 0.0,\n        \"max\": 370621.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
17.337371897437,\n          16.9,\n          370621.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"open_acc\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
131027.21602194426,\n        \"min\": 1.0,\n        \"max\":
370621.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
11.393337128764966,\n          11.0,\n          370621.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"pub_rec\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
131029.93581755321,\n        \"min\": 0.0,\n        \"max\":
370621.0,\n        \"num_unique_values\": 5,\n        \"samples\": [\n
0.1722784191937316,\n          86.0,\n          0.5236205359181035\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"revol_bal\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
607029.0923187542,\n        \"min\": 0.0,\n        \"max\":
1743266.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\
n          15950.622946352203,\n          11303.0,\n
370621.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"revol_util\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 130977.33713317609,\n        \"min\":
0.0,\n        \"max\": 370621.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          53.991221868161816,\n          55.0,\n
370621.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"total_acc\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 131021.0156939621,\n        \"min\":
2.0,\n        \"max\": 370621.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          25.51850272920315,\n          24.0,\n
370621.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"mort_acc\",\n      \"properties\": {\n        \"dtype\":
```

\"number\",\n   \"std\": 131032.19862752155,\n   \"min\":
0.0,\n   \"max\": 370621.0,\n   \"num_unique_values\": 7,\n
\"samples\": [\n   370621.0,\n   1.7779645513880757,\n
3.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"pub_rec_bankruptcies\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 131033.8834960711,\n   \"min\":
0.0,\n   \"max\": 370621.0,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   0.11733819724192639,\n   8.0,\n
0.3508556132077476\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

Insights

There is significant difference found in the mean and median of the following

attributes

loan_amnt

terms

installment

revol_bal etc.

These attributes might contain outliers

```
df.describe(include = 'object')
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 4,\n \"fields\": [\n
{\n   \"column\": \"term\",\n   \"properties\": {\n
\"dtype\": \"string\",\n   \"num_unique_values\": 4,\n
\"samples\": [\n   2,\n   \"280484\",\n
\"370621\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"grade\",\n   \"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n   \"samples\": [\n   7,\n
\"108666\",\n   \"370621\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"sub_grade\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n   \"samples\": [\n   35,\n
\"24960\",\n   \"370621\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"emp_title\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n   \"samples\": [\n   172143,\n
\"4367\",\n   \"370621\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"emp_length\",\n

```
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          11,\n
\"123389\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"home_ownership\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          6,\n
\"186218\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"verification_status\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          3,\n
\"125225\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"issue_d\",\n      \"properties\":
{\n        \"dtype\": \"date\",\n        \"min\": \"1970-01-01
00:00:00.000000111\",\n        \"max\": \"2014-10-01 00:00:00\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          111,\n
\"14133\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"loan_status\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          2,\n
\"299363\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"purpose\",\n      \"properties\":
{\n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          14,\n          \"221262\",\n
\"370621\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"title\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          45474,\n
\"144609\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"earliest_cr_line\",\n
\"properties\": {\n        \"dtype\": \"date\",\n        \"min\":
\"1970-01-01 00:00:00.000000664\",\n        \"max\": \"2000-10-01
00:00:00\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          664,\n          \"2861\",\n          \"370621\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"initial_list_status\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          2,\n          \"222308\",\n          \"370621\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"application_type\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          3,\n          \"370065\",\n          \"370621\"\
```

```
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"address\",\n      \"properties\": {\n        \"dtype\": \"string\",\
n        \"num_unique_values\": 4,\n        \"samples\": [\n
368575,\n          \"8\",\n          \"370621\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


Insights

Most of the loan disburesed for the 36 months period

Most of the loan applicant have mortgage the home

Majority of loans been fully paid off

Majorily the loans been disbursed for the purpose of debt consolidation

Most of the applicant is Individual

```
df.describe(include='all').transpose()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 27,\n  \"fields\": [\n
{\n      \"column\": \"count\",\n      \"properties\": {\n
\"dtype\": \"date\",\n        \"min\": 370621.0,\n        \"max\":
370621.0,\n        \"num_unique_values\": 1,\n        \"samples\": [\n
370621.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"unique\",\n      \"properties\": {\n        \"dtype\": \"date\",\n
\"min\": 2,\n        \"max\": 368575,\n        \"num_unique_values\":
12,\n        \"samples\": [\n          664\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"top\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 15,\n
\"samples\": [\n          \"debt_consolidation\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"freq\",\n      \"properties\": {\n
\"dtype\": \"date\",\n        \"min\": \"8\",\n        \"max\":
\"370065\",\n        \"num_unique_values\": 15,\n        \"samples\":
[\n          \"221262\"\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"mean\",\n      \"properties\": {\n        \"dtype\":
\"date\",\n        \"min\": 0.11733819724192639,\n        \"max\":
75187.8719837516,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          1.7779645513880757\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"std\",\n      \"properties\": {\n
\"dtype\": \"date\",\n        \"min\": 0.3508556132077476,\n
\"max\": 62090.2567072765,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          2.0583086519715392\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
```

```
n    },\n    {\n      \"column\": \"min\",\n      \"properties\": {\n
\"dtype\": \"date\",\n        \"min\": 0.0,\n        \"max\": 4000.0,\
n        \"num_unique_values\": 7,\n        \"samples\": [\n
500.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"25%\",\n      \"properties\": {\n        \"dtype\": \"date\",\n
\"min\": 0.0,\n        \"max\": 46000.0,\n
\"num_unique_values\": 10,\n        \"samples\": [\n          36.1\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"50%\",\n      \"properties\": {\
n        \"dtype\": \"date\",\n        \"min\": 0.0,\n        \"max\":
65000.0,\n        \"num_unique_values\": 11,\n        \"samples\": [\n
11.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"75%\",\n      \"properties\": {\n        \"dtype\": \"date\",\n
\"min\": 0.0,\n        \"max\": 90000.0,\n
\"num_unique_values\": 11,\n        \"samples\": [\n          14.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"max\",\n      \"properties\": {\
n        \"dtype\": \"date\",\n        \"min\": 8.0,\n        \"max\":
8706582.0,\n        \"num_unique_values\": 12,\n        \"samples\":
[\n          34.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Insights

Outliers: The significant differences between mean & median in key attributes like loan amount and revolving balance indicate potential outliers.

Loan Duration Preference: A preference for 36-month loan terms among borrowers suggests a balance between manageable installments.

Home Ownership Trends: The prevalence of applicants with mortgaged homes suggests financial stability or a need for substantial, property-secured loans.

Successful Loan Repayment: Most loans being fully paid off reflects positively on borrowers' financial commitment, indicating effective lending criteria.

Debt Consolidation Dominance: The primary use of loans for debt consolidation highlights a common strategy to manage or reduce high-interest debt.

Individual Borrowers: The predominance of individual applicants suggests that personal loans are a major market segment.

```
n_columns = df.select_dtypes('float64').columns.tolist()

n_columns
```

```
['loan_amnt',
 'int_rate',
 'installment',
```

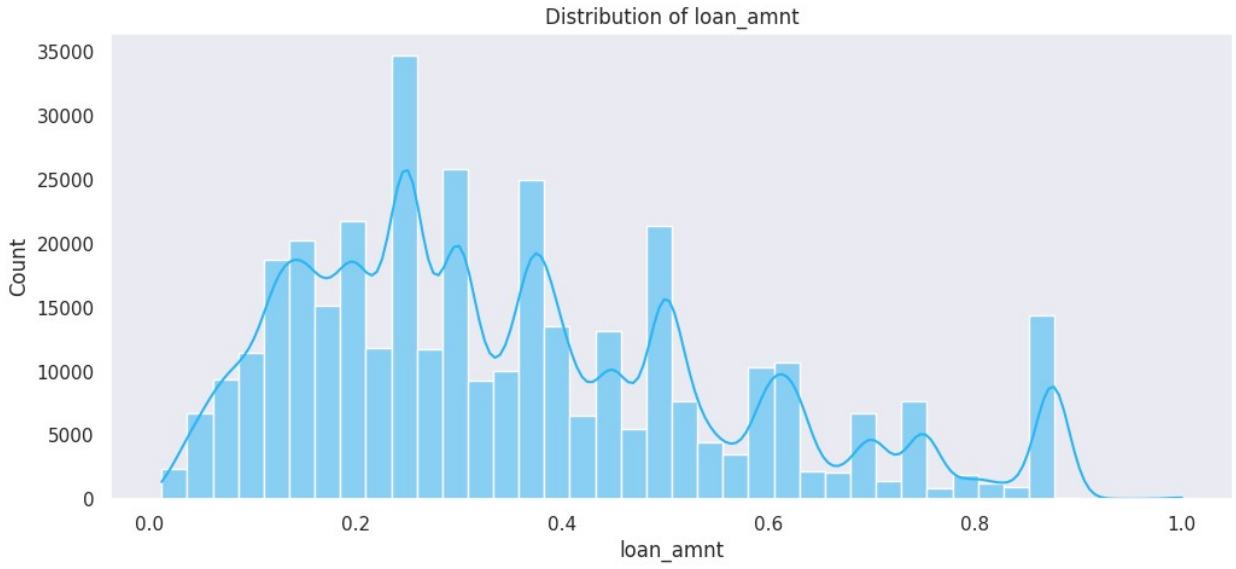
```
 'annual_inc',
 'dti',
 'open_acc',
 'pub_rec',
 'revol_bal',
 'revol_util',
 'total_acc',
 'mort_acc',
 'pub_rec_bankruptcies']
```

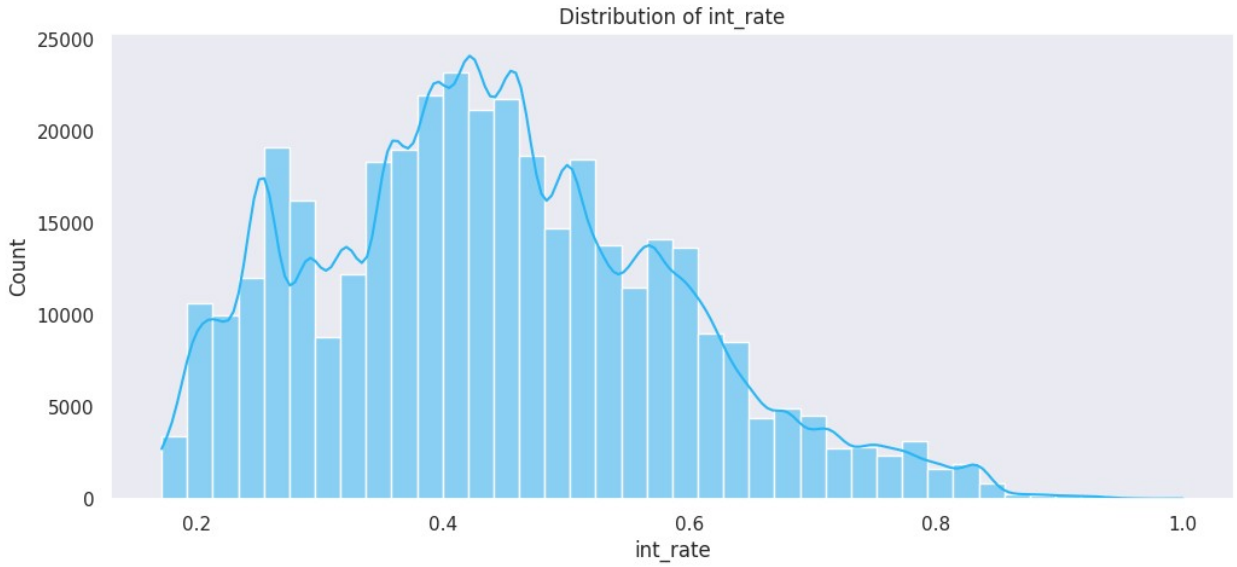
```
c_columns = ['home_ownership', 'verification_status', 'loan_status',
'application_type', 'grade', 'sub_grade', 'term']
c_columns
```

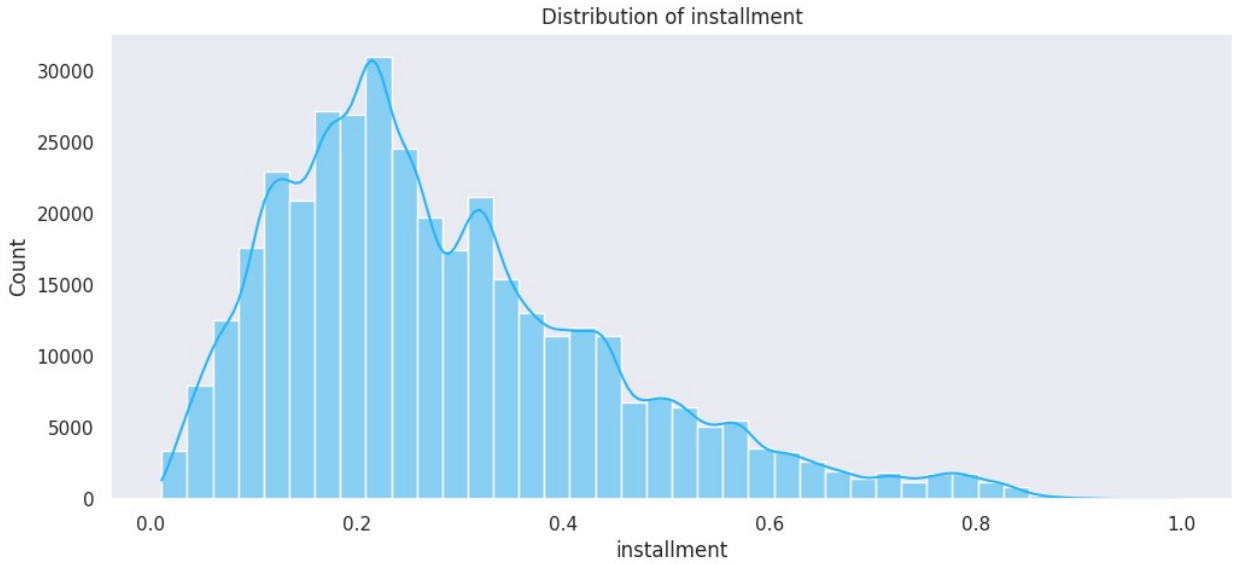
```
['home_ownership',
 'verification_status',
 'loan_status',
 'application_type',
 'grade',
 'sub_grade',
 'term']
```

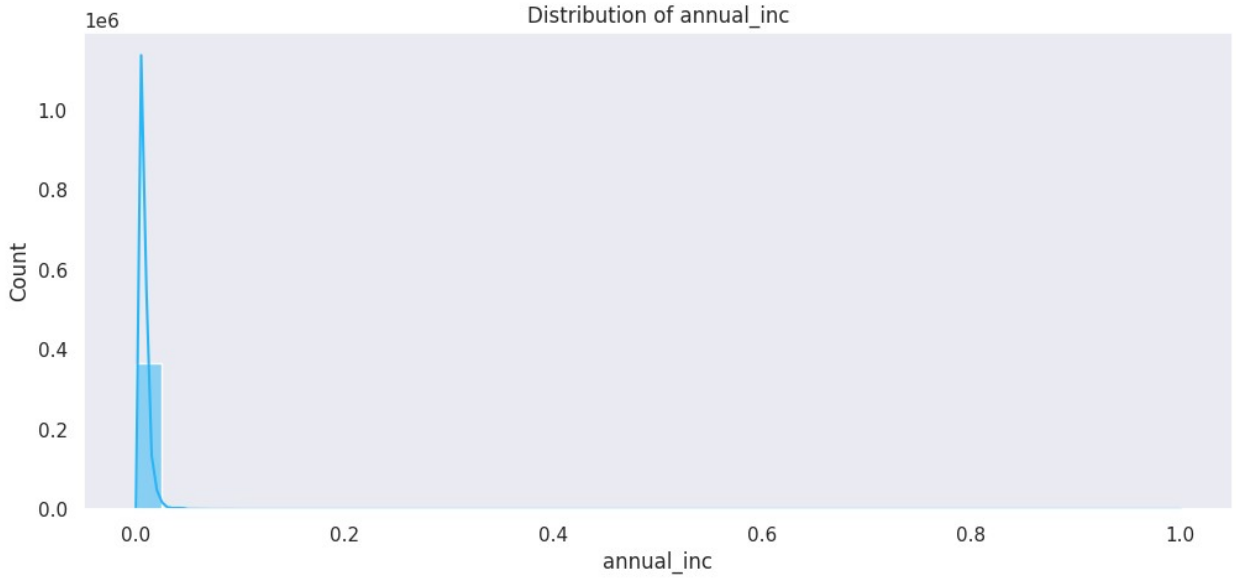
Univariate Analysis

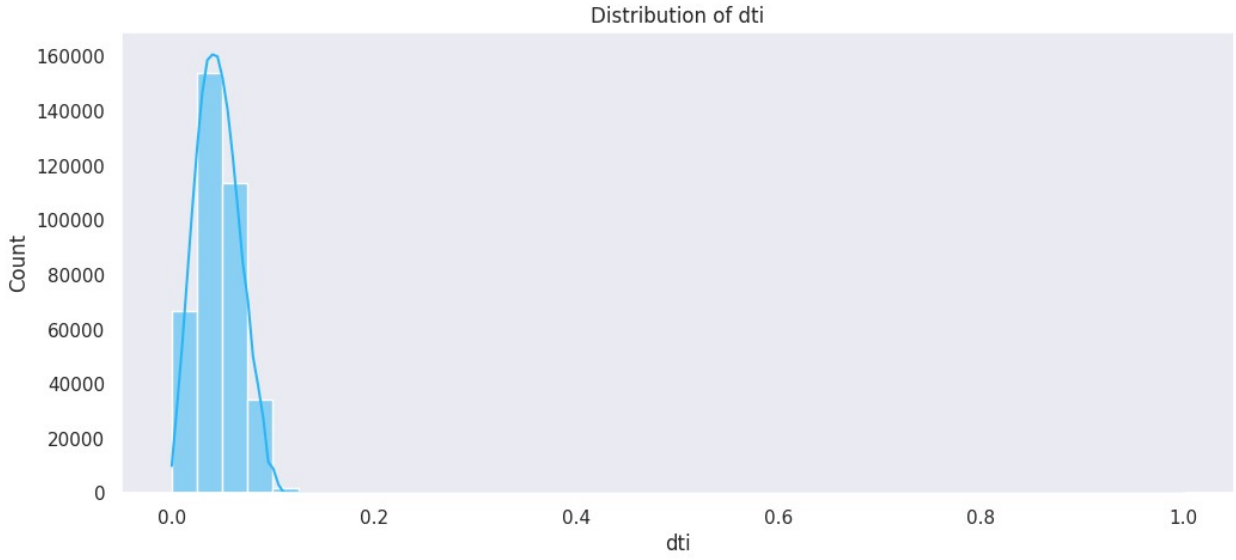
```
for i in n_columns:
    plt.figure(figsize=(12,5))
    plt.title("Distribution of {}".format(i))
    sns.histplot(df[i]/df[i].max(), kde=True,color="#29B6F6", bins=40)
    plt.show()
```

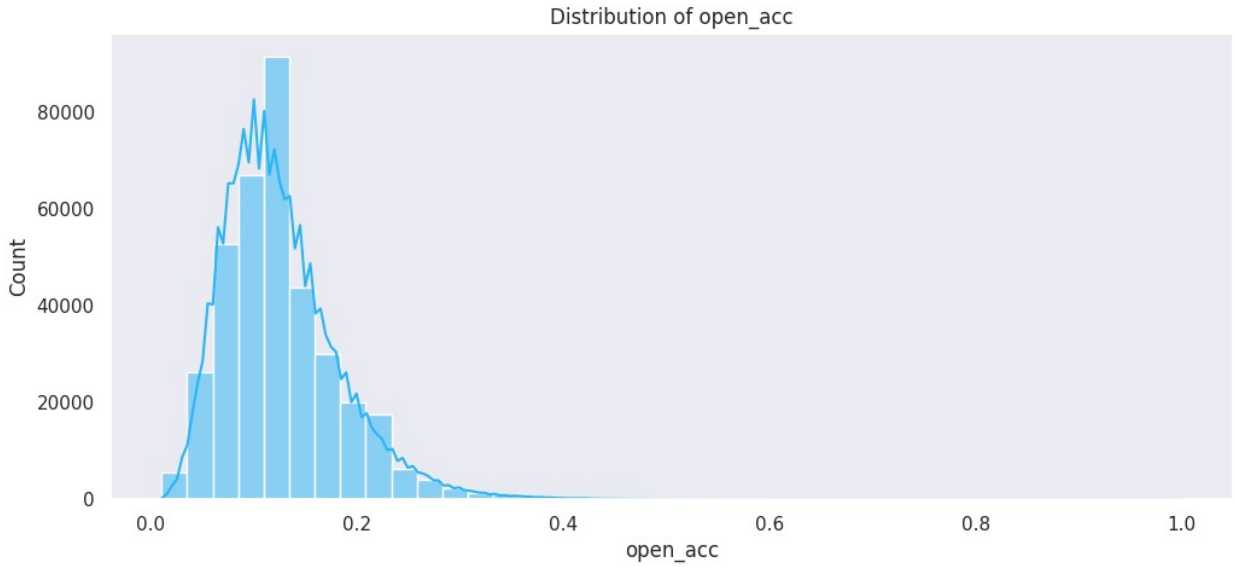


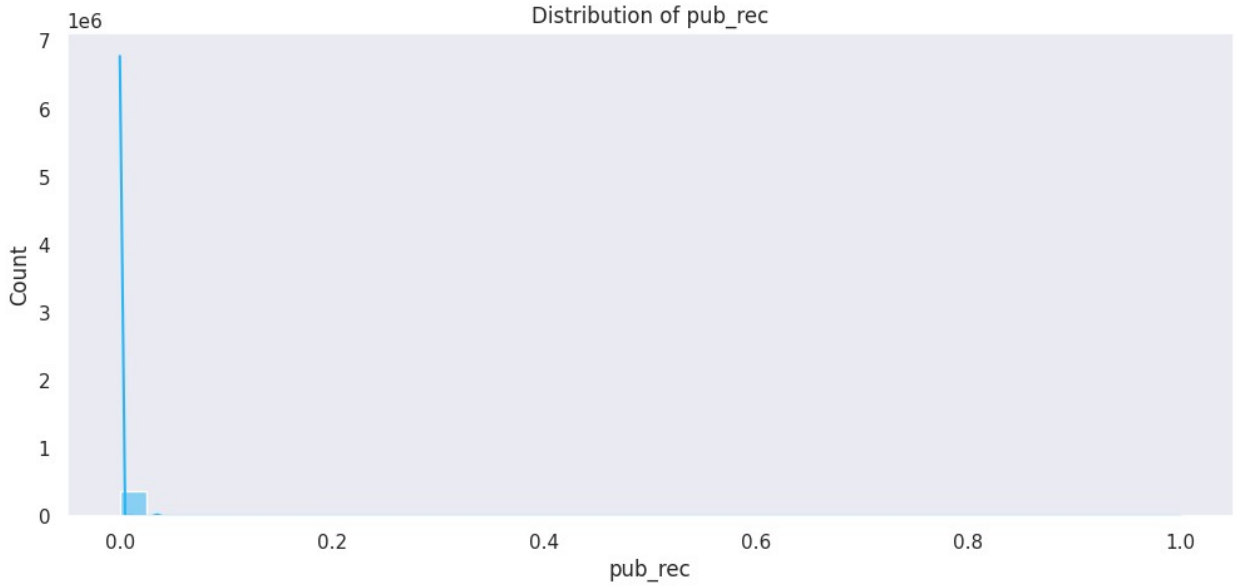

Distribution of int_rate
Count
25000
20000
15000
10000
5000
0
0.2
0.4
0.6
0.8
1.0
int_rate

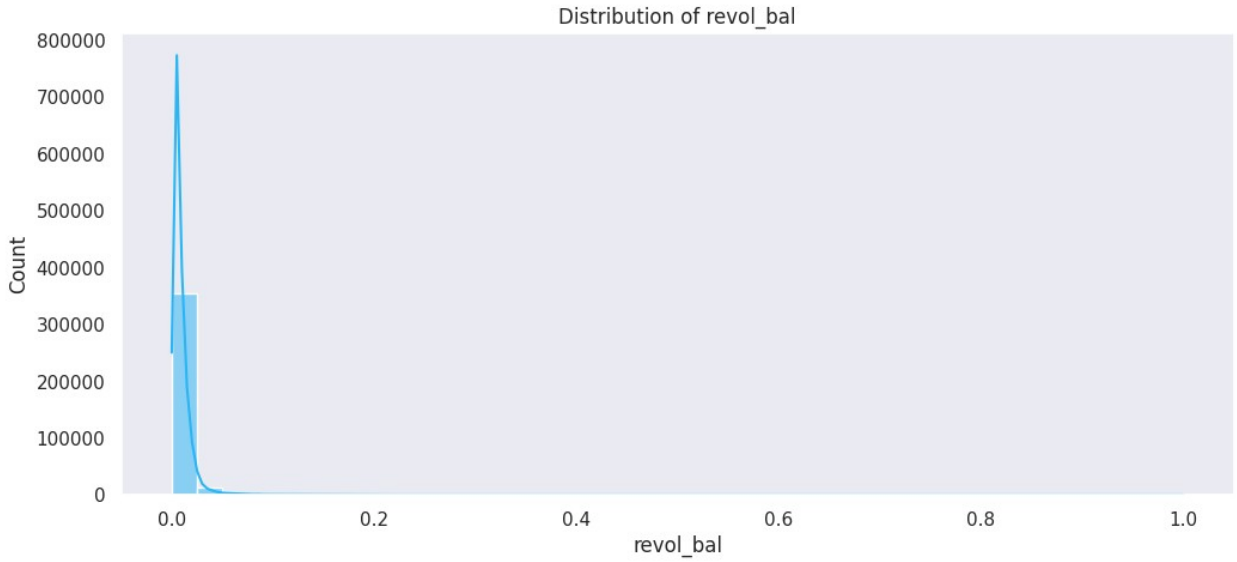

Distribution of installment
Count
30000
25000
20000
15000
10000
5000
0
0.0
0.2
0.4
0.6
0.8
1.0
installment

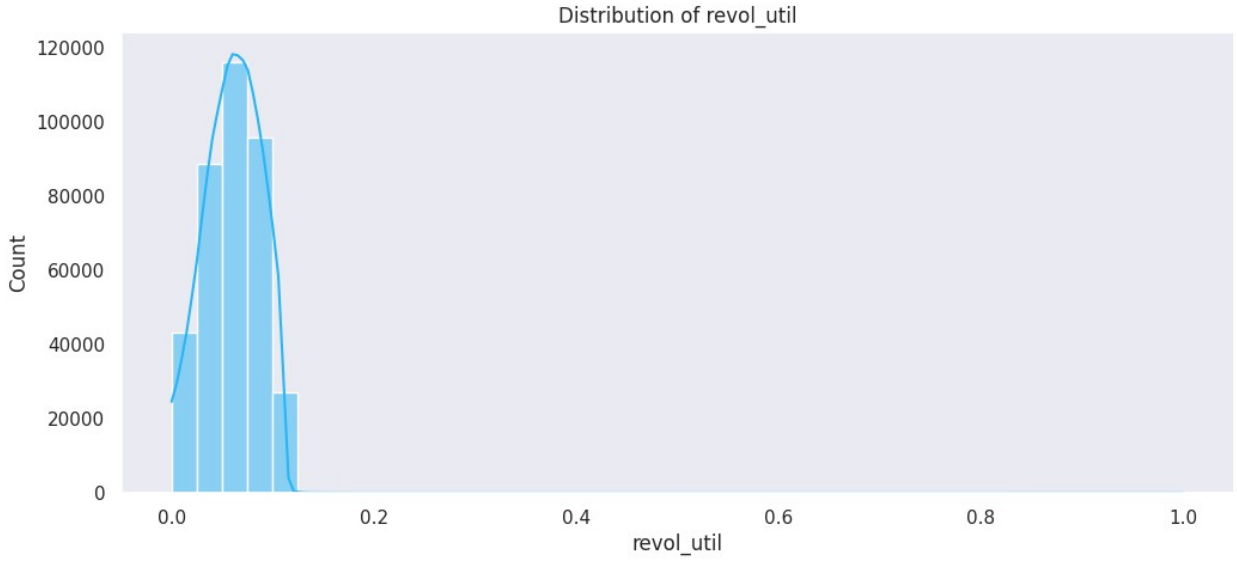

Distribution of annual_inc
1e6
1.0
0.8
0.6
0.4
0.2
0.0
Count
0.0
0.2
0.4
0.6
0.8
1.0
annual_inc

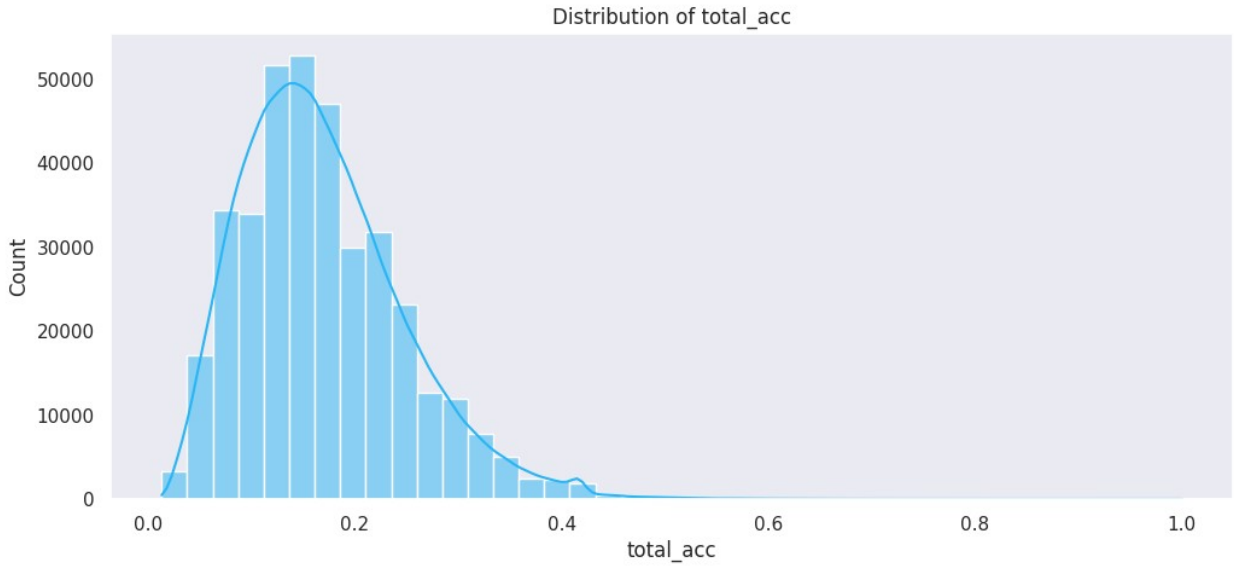

Distribution of dti
160000
140000
120000
100000
80000
60000
40000
20000
0
Count
0.0
0.2
0.4
0.6
0.8
1.0
dti

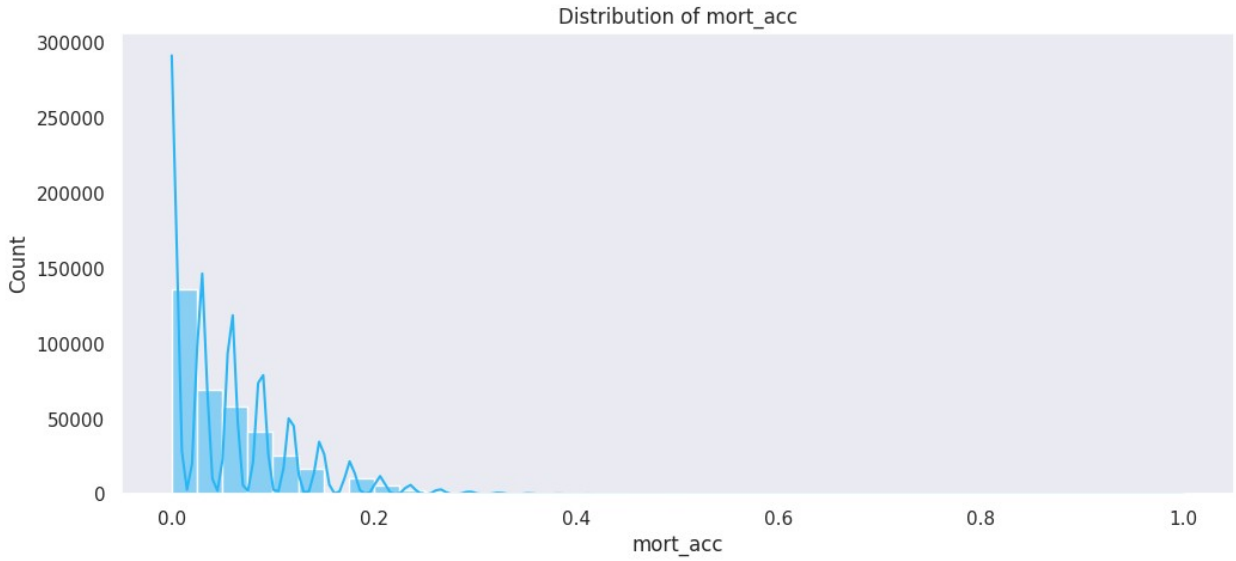

Distribution of open_acc
Count
80000
60000
40000
20000
0
0.0
0.2
0.4
0.6
0.8
1.0
open_acc

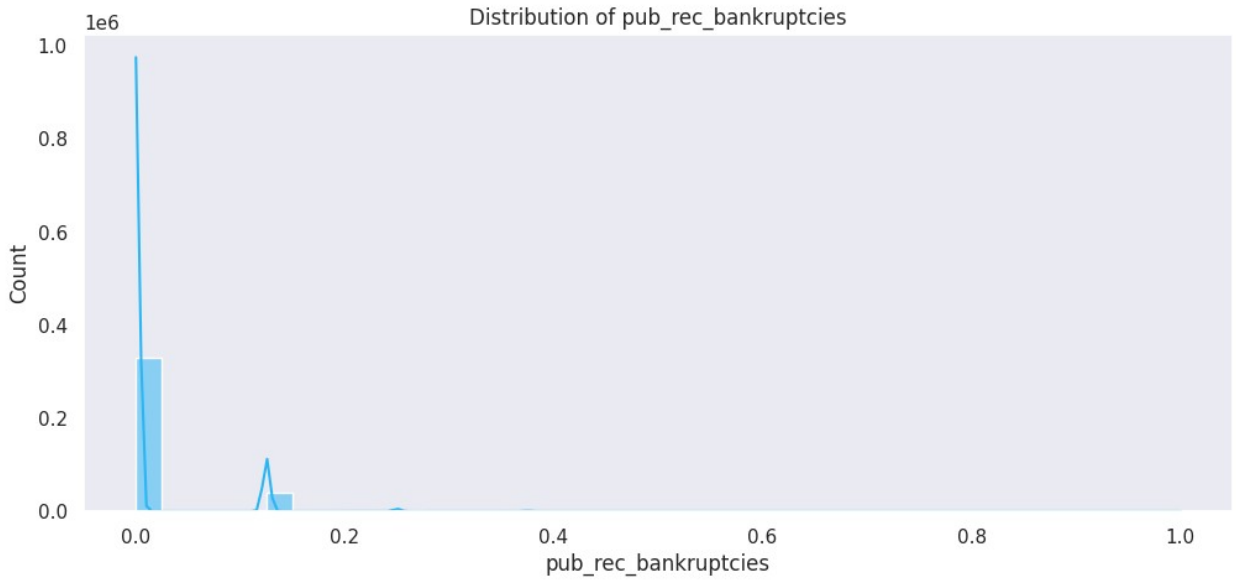

Distribution of pub_rec
1e6
Count
7
6
5
4
3
2
1
0
0.0
0.2
0.4
0.6
0.8
1.0
pub_rec


Distribution of revol_bal
800000
700000
600000
500000
400000
300000
200000
100000
0
Count
0.0
0.2
0.4
0.6
0.8
1.0
revol_bal


Distribution of revol_util
120000
100000
80000
60000
40000
20000
0
Count
0.0
0.2
0.4
0.6
0.8
1.0
revol_util

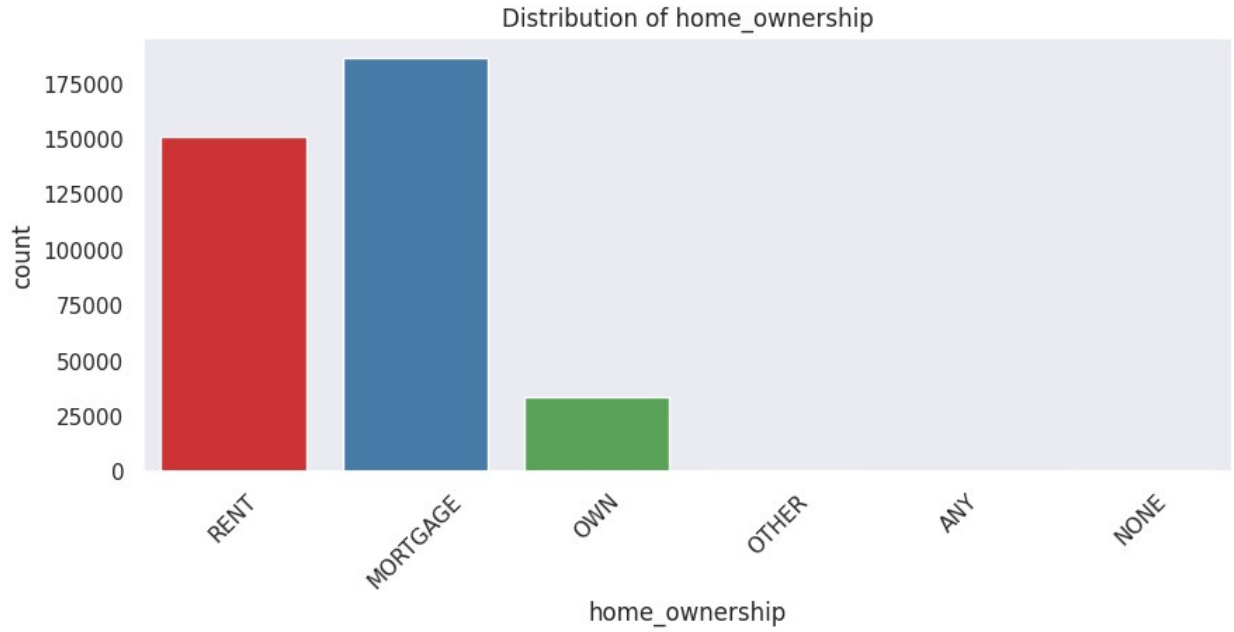

Distribution of total_acc
Count
50000
40000
30000
20000
10000
0
0.0
0.2
0.4
0.6
0.8
1.0
total_acc


Distribution of mort_acc
Count
300000
250000
200000
150000
100000
50000
0
0.0
0.2
0.4
0.6
0.8
1.0
mort_acc

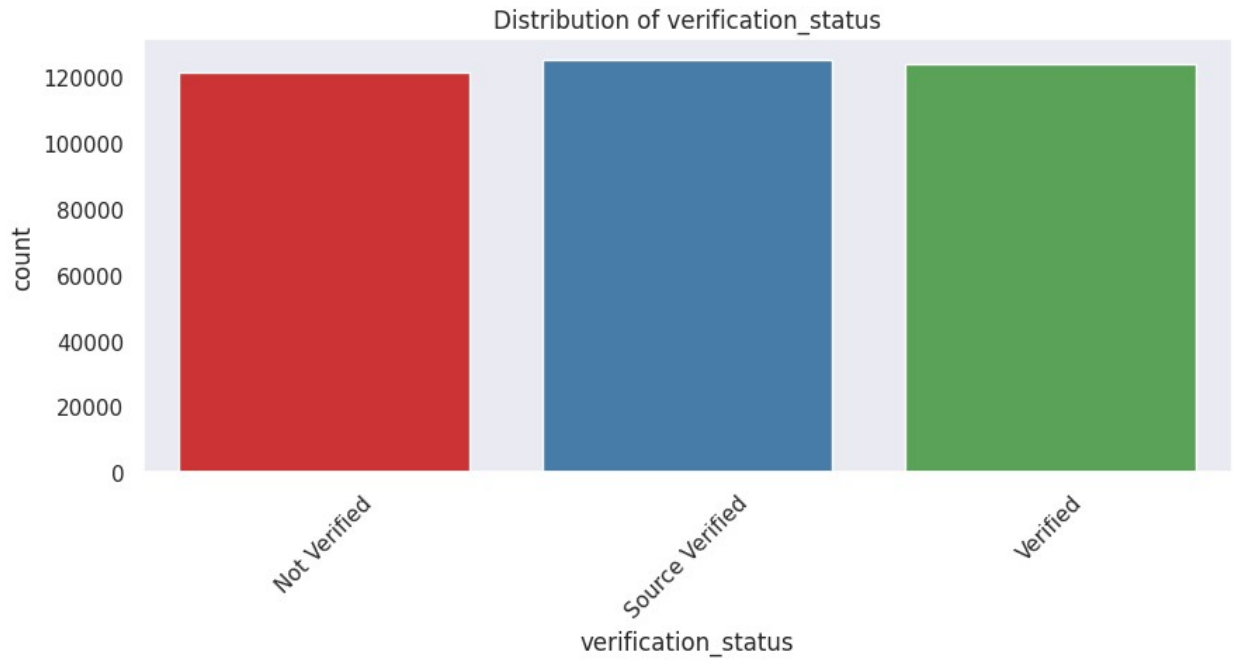



```
custom_palette = sns.color_palette("Set1", 8)

for i in c_columns:
    plt.figure(figsize=(10, 4))
    sns.set(style="dark")
    plt.title(f'Distribution of {i}')
    sns.countplot(data=df, x=i, palette=custom_palette)
    plt.xticks(rotation=45)
    plt.show()
```

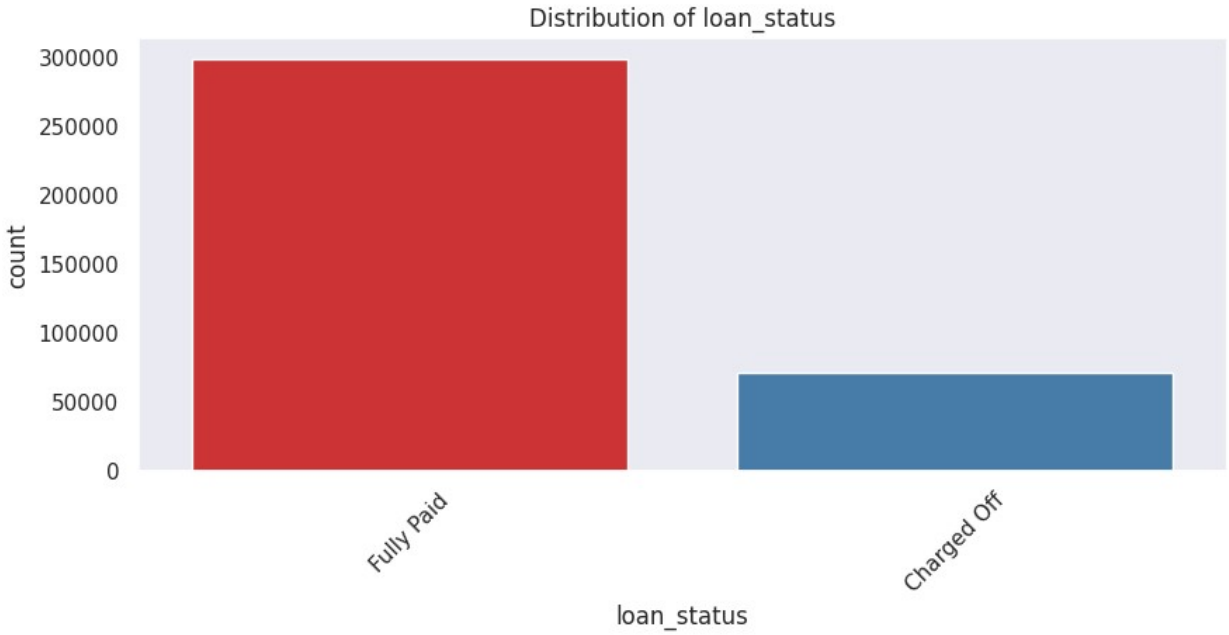
```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning: The palette list has
more values (8) than needed (6), which may not be intended.
  sns.countplot(data=df, x=i, palette=custom_palette)
```



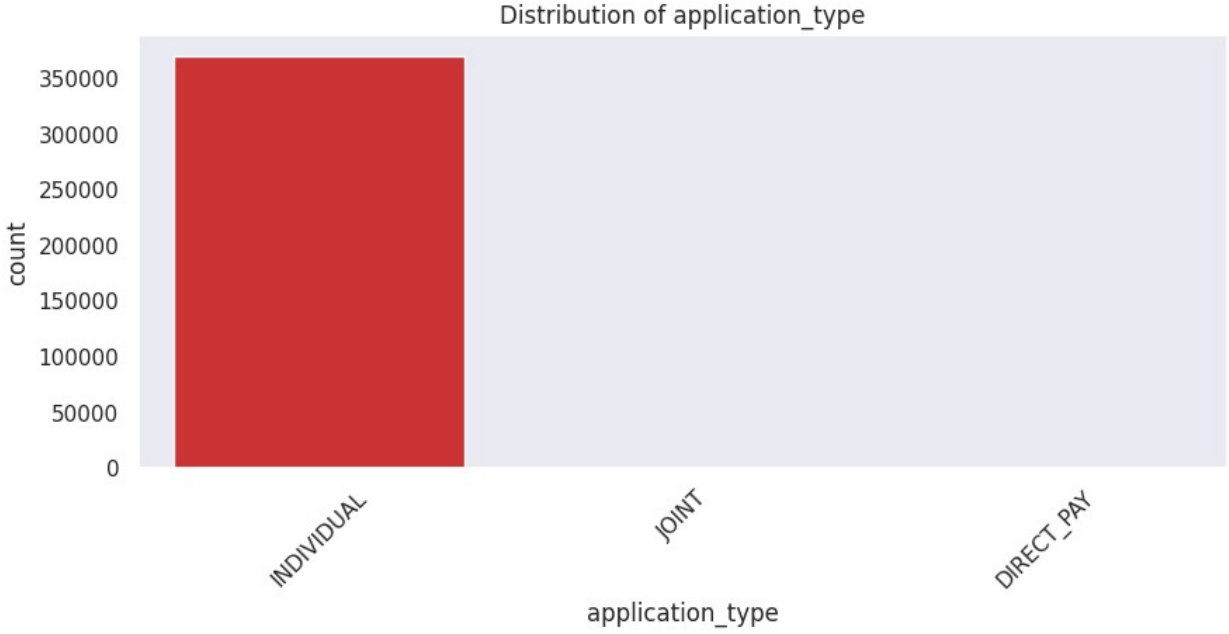

```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning: The palette list has
more values (8) than needed (3), which may not be intended.
  sns.countplot(data=df, x=i, palette=custom_palette)
```



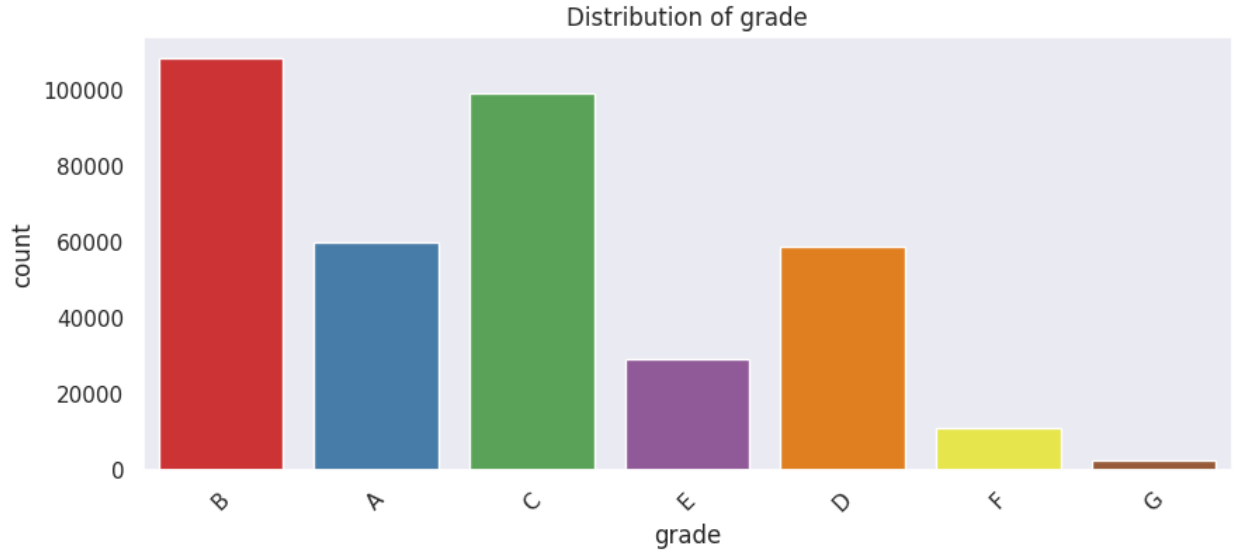

```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning: The palette list has
more values (8) than needed (2), which may not be intended.
  sns.countplot(data=df, x=i, palette=custom_palette)
```



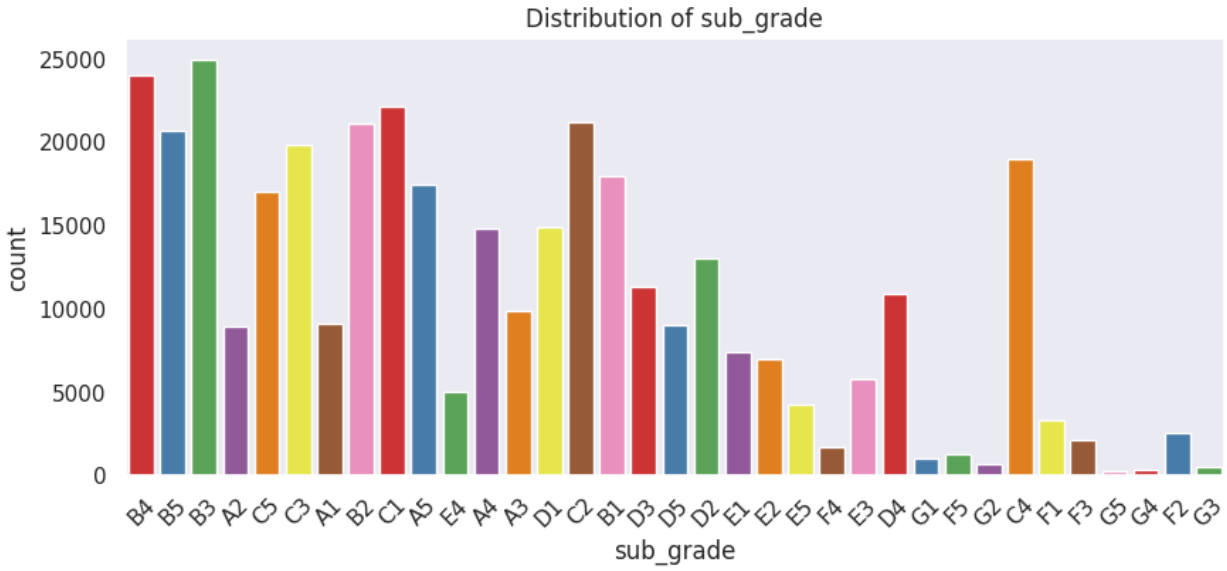

```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning: The palette list has
more values (8) than needed (3), which may not be intended.
  sns.countplot(data=df, x=i, palette=custom_palette)
```



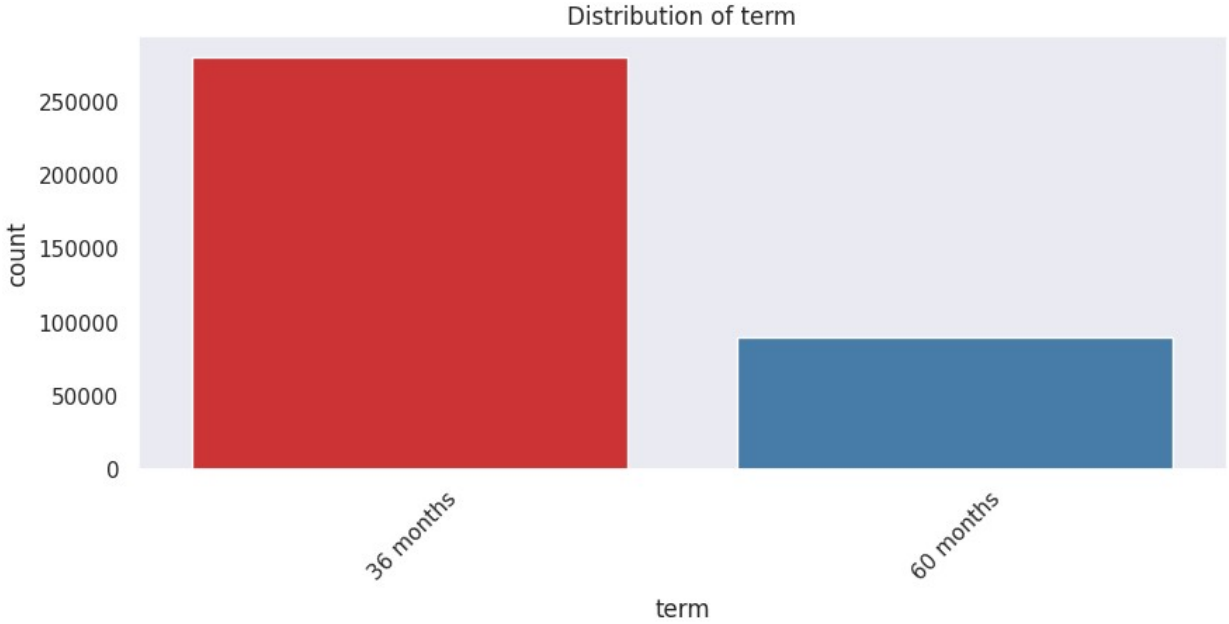
```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning: The palette list has
more values (8) than needed (7), which may not be intended.
  sns.countplot(data=df, x=i, palette=custom_palette)
```




```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning:
The palette list has fewer values (8) than needed (35) and will cycle,
which may produce an uninterpretable plot.
  sns.countplot(data=df, x=i, palette=custom_palette)
```



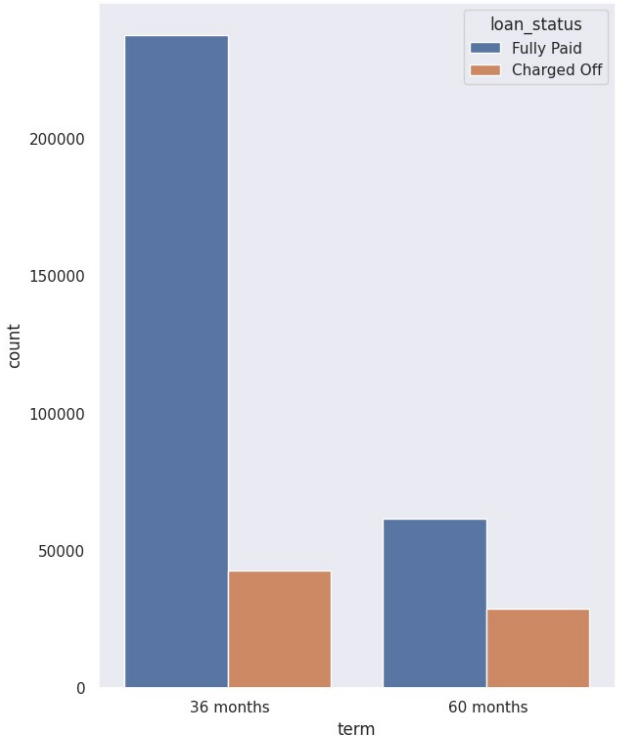

```
<ipython-input-98-496c4d8e9b0e>:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(data=df, x=i, palette=custom_palette)
<ipython-input-98-496c4d8e9b0e>:7: UserWarning: The palette list has
more values (8) than needed (2), which may not be intended.
  sns.countplot(data=df, x=i, palette=custom_palette)
```

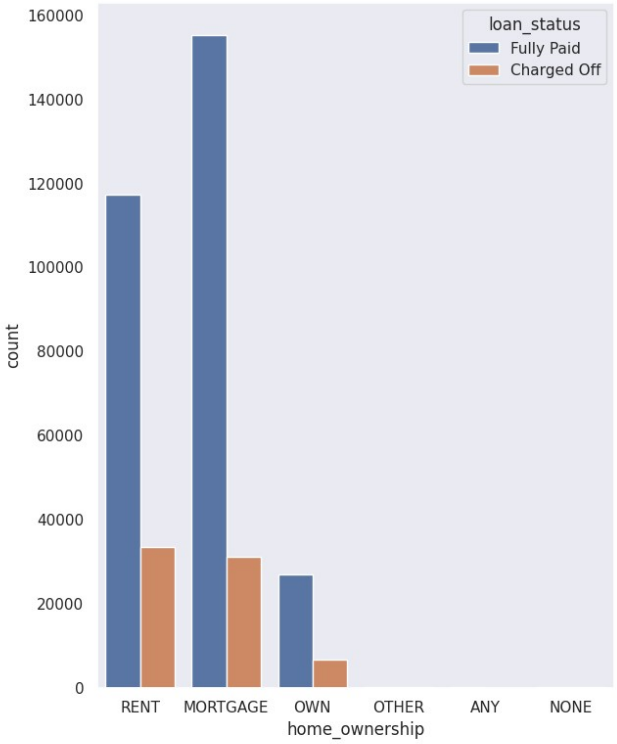


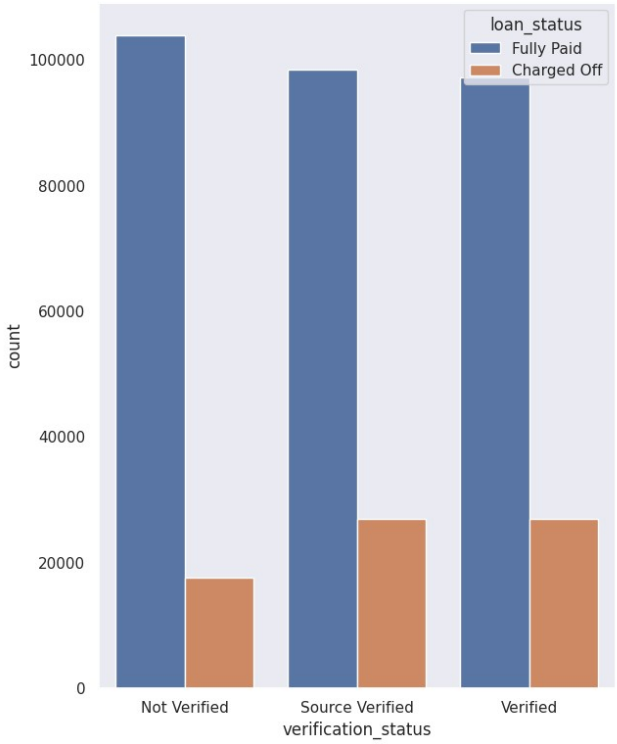
Bivariate Analysis

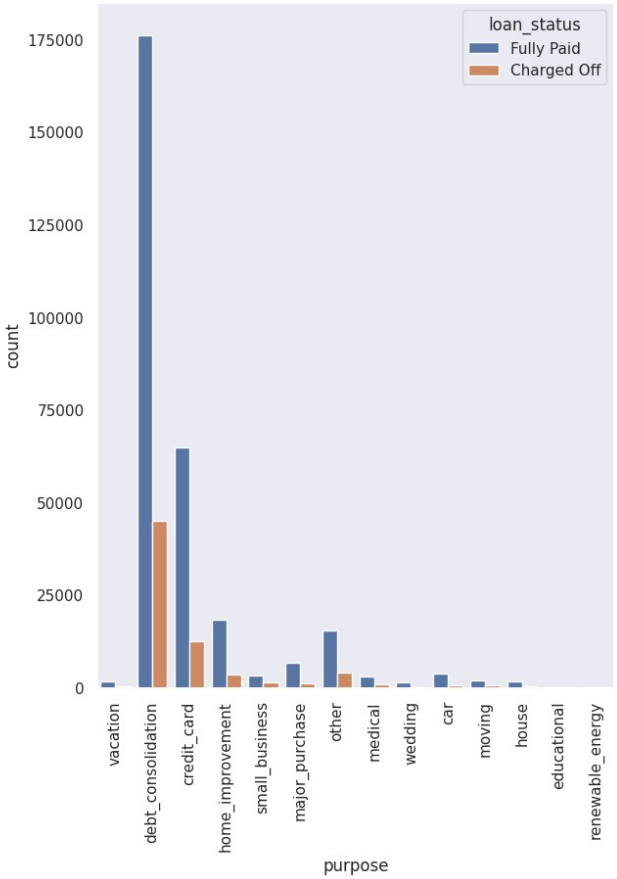
```
plt.figure(figsize=(15,20))

plt.subplot(2,2,1)
sns.countplot(x='term',data=df,hue='loan_status')

plt.subplot(2,2,2)
sns.countplot(x='home_ownership',data=df,hue='loan_status')

plt.subplot(2,2,3)
sns.countplot(x='verification_status',data=df,hue='loan_status')

plt.subplot(2,2,4)
g=sns.countplot(x='purpose',data=df,hue='loan_status')
g.set_xticklabels(g.get_xticklabels(),rotation=90)

plt.show()
```

```
<ipython-input-99-5431175f47c8>:14: UserWarning: set_ticklabels()
should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
  g.set_xticklabels(g.get_xticklabels(),rotation=90)
```


loan_status
Fully Paid
Charged Off
count
200000
150000
100000
50000
0
36 months
60 months
term

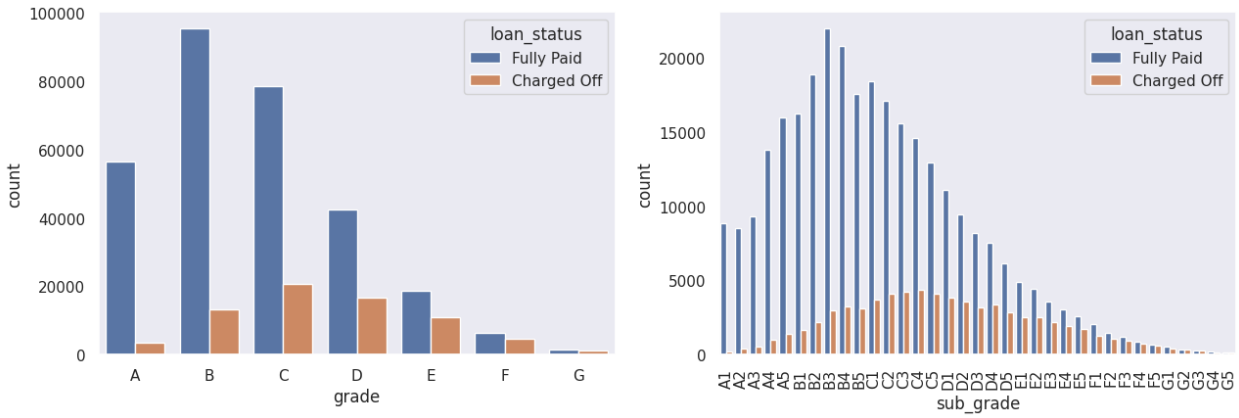

loan_status
Fully Paid
Charged Off
count
160000
140000
120000
100000
80000
60000
40000
20000
0
RENT
MORTGAGE
OWN
OTHER
ANY
NONE
home_ownership


loan_status
Fully Paid
Charged Off
count
100000
80000
60000
40000
20000
0
Not Verified
Source Verified
Verified
verification_status


loan_status
Fully Paid
Charged Off
count
175000
150000
125000
100000
75000
50000
25000
0
vacation
debt_consolidation
credit_card
home_improvement
small_business
major_purchase
other
medical
wedding
car
moving
house
educational
renewable_energy
purpose

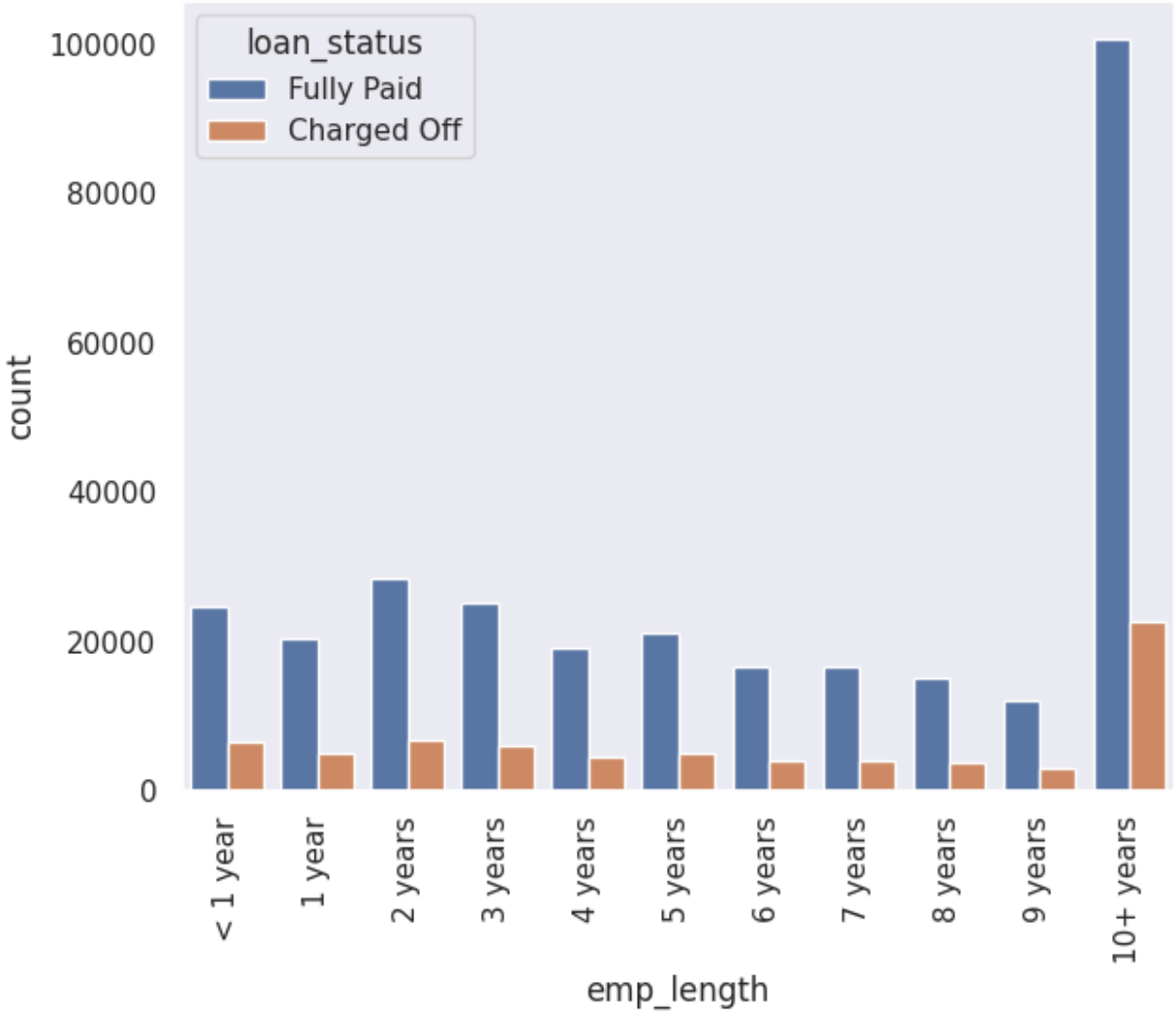
```
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
grade = sorted(df.grade.unique().tolist())
sns.countplot(x='grade', data=df, hue='loan_status', order=grade)

plt.subplot(2, 2, 2)
sub_grade = sorted(df.sub_grade.unique().tolist())
g = sns.countplot(x='sub_grade', data=df, hue='loan_status',
order=sub_grade)
g.set_xticklabels(g.get_xticklabels(), rotation=90)

plt.show()
```

```
<ipython-input-100-4fc383fe14dc>:10: UserWarning: set_ticklabels()
should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
  g.set_xticklabels(g.get_xticklabels(), rotation=90)
```




```
plt.figure(figsize=(15,12))

plt.subplot(2,2,1)
order = ['< 1 year', '1 year', '2 years', '3 years', '4 years', '5
years',
         '6 years', '7 years', '8 years', '9 years', '10+ years',]
g=sns.countplot(x='emp_length',data=df,hue='loan_status',order=order)
g.set_xticklabels(g.get_xticklabels(),rotation=90)

plt.show()
```

```
<ipython-input-101-955bc5b8fec7>:7: UserWarning: set_ticklabels()
should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
  g.set_xticklabels(g.get_xticklabels(),rotation=90)
```



Insights and Key Findings

Loan Duration Preferences:

A significant majority of borrowers prefer 36-month loan terms, which also exhibit higher completion and repayment rates compared to 60-month loans.

Popular Loan Purposes:

The most frequently requested loan types are mortgage-related and rental loans, followed closely by debt consolidation loans, indicating a strong demand for refinancing and housing support.

Credit Grade & Repayment Performance:

Borrowers with a credit grade of 'B', particularly subgrade 'B3', consistently show the highest repayment reliability, making them a lower-risk segment for lending.

Profession-Based Approval Trends:

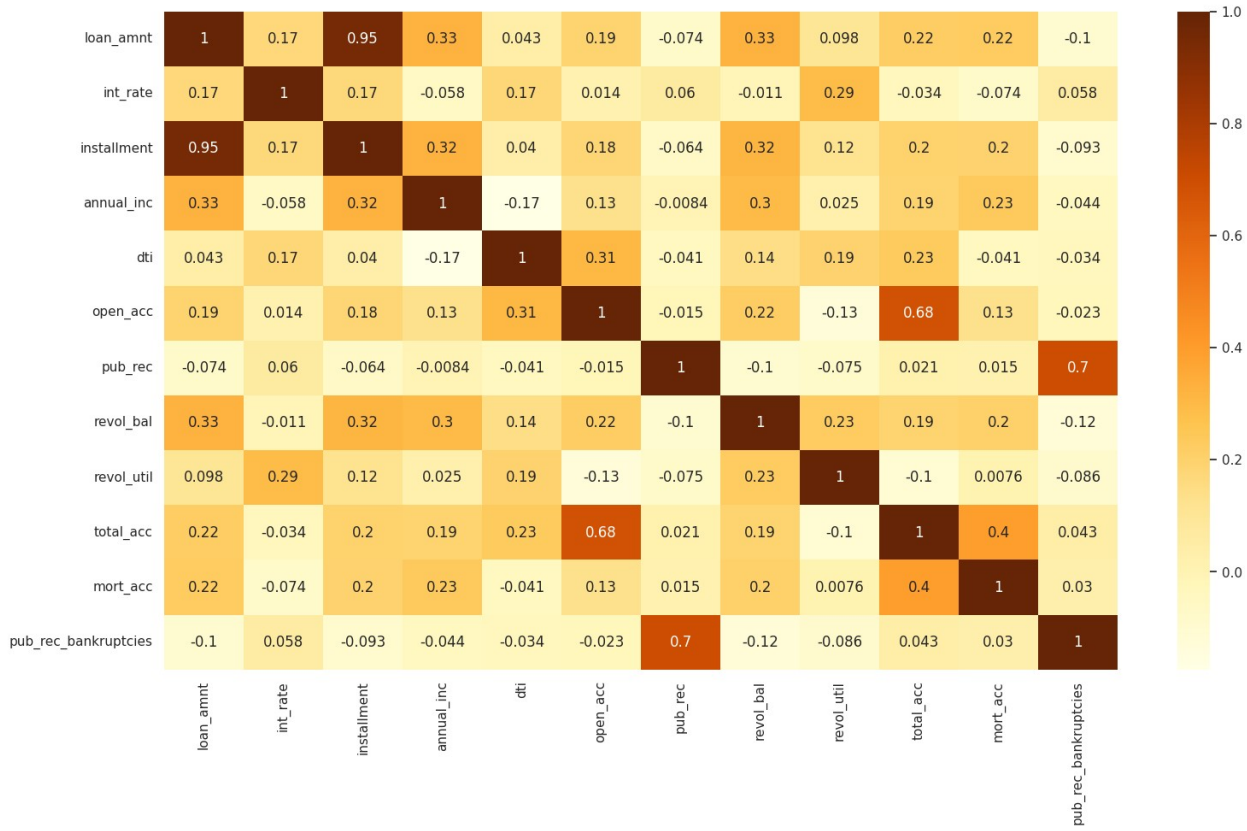
Managers and teachers are among the top professions with the highest loan approval rates, suggesting a correlation between job stability and creditworthiness.

Employment Tenure & Repayment Behavior:

Individuals with over 10 years of employment history demonstrate strong repayment behavior, indicating that longer job tenure is a good predictor of loan reliability.

Correlation Analysis

```
plt.figure(figsize=(18,10))
sns.heatmap(df.corr(numeric_only=True), cmap = 'YlOrBr', annot = True)
plt.show()
```




```
df.corr(numeric_only=True)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 12,\n  \"fields\": [\n
{\n      \"column\": \"loan_amnt\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.35168433858260084,\n
\"min\": -0.10049810172929258,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.22498776020495778,\n          0.22136423409839157,\n          1.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"int_rate\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```
0.2911130952728497,\n        \"min\": -0.07422124737963509,\n
\"max\": 1.0,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          -0.07422124737963509,\n          -
0.03384334476960896,\n          0.17355737554246808\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"installment\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.35037698553043534,\n        \"min\": -0.09309261663990022,\n
\"max\": 1.0,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          0.19668619619229163,\n
0.1997920146135309,\n          0.953122422565513\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"annual_inc\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.3051563571292586,\n        \"min\": -0.17447370401178203,\n
\"max\": 1.0,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          0.23498989667868714,\n
0.1908979331451691,\n          0.32732008461923406\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"dti\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.3002135603579252,\n
\"min\": -0.17447370401178203,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n          -
0.04135320895328787,\n          0.229597819338884,\n
0.04283640785269032\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"open_acc\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.32008490169181886,\n        \"min\": -
0.13146963884260265,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.12982268038586264,\n          0.6810777847011693,\n
0.1923063389144639\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"pub_rec\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 0.3507194976303664,\n        \"min\": -
0.10091469704106142,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.015448878159446794,\n          0.02111363628166752,\n          -
0.07365740803179928\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"revol_bal\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.28849590810617626,\n        \"min\": -
0.12274658621013457,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.20257479803040468,\n          0.1923731688158834,\n
0.3278086129883803\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"revol_util\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.3067594289380684,\n        \"min\": -
```

```
0.13146963884260265,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.007626553707596485,\n          -0.10276142545829496,\n
0.09844623537161389\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"total_acc\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.313441984315257,\n        \"min\": -
0.10276142545829496,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.3975109366993595,\n          1.0,\n          0.22136423409839157\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"mort_acc\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.2889779524564675,\n        \"min\": -0.07422124737963509,\n
\"max\": 1.0,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          1.0,\n          0.3975109366993595,\n
0.22498776020495778\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"pub_rec_bankruptcies\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.3558274523634983,\n        \"min\": -
0.12274658621013457,\n        \"max\": 1.0,\n
\"num_unique_values\": 12,\n        \"samples\": [\n
0.030445983917399985,\n          0.04315543198129393,\n          -
0.10049810172929258\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Insights

Loan Amount & Annual Income

There is a moderate positive correlation between loan_amnt and annual_inc (0.44). This suggests that individuals with higher income tend to apply for, and are approved for, larger loan amounts.

Loan Amount & Installment Amount

A very strong positive correlation exists between loan_amnt and installment (0.95). Naturally, larger loans result in higher monthly installments.

Loan Amount & Credit Activity

There is a weak to moderate positive correlation between loan_amnt and both total_acc (0.21) and mort_acc (0.21). Borrowers with more credit accounts, including mortgages, may be seen as more financially established and eligible for higher loan amounts.

Interest Rate & Annual Income

A weak negative correlation is observed between int_rate and annual_inc (-0.10). This indicates that borrowers with higher income may qualify for lower interest rates, possibly due to better financial stability and lower risk.

Annual Income & Credit Accounts

annual_inc is moderately correlated with total_acc (0.28) and mort_acc (0.32), suggesting that high-income individuals typically hold more credit accounts and mortgage obligations.

Revolving Balance & Revolving Utilization

There is a moderate positive correlation (0.37) between revol_bal and revol_util, showing that borrowers with higher balances are also utilizing a larger portion of their credit lines — which could indicate higher credit dependence.

Open Accounts & Total Accounts

A strong correlation (0.65) exists between open_acc and total_acc, which is expected — borrowers with more open credit lines tend to have a higher total number of credit accounts.

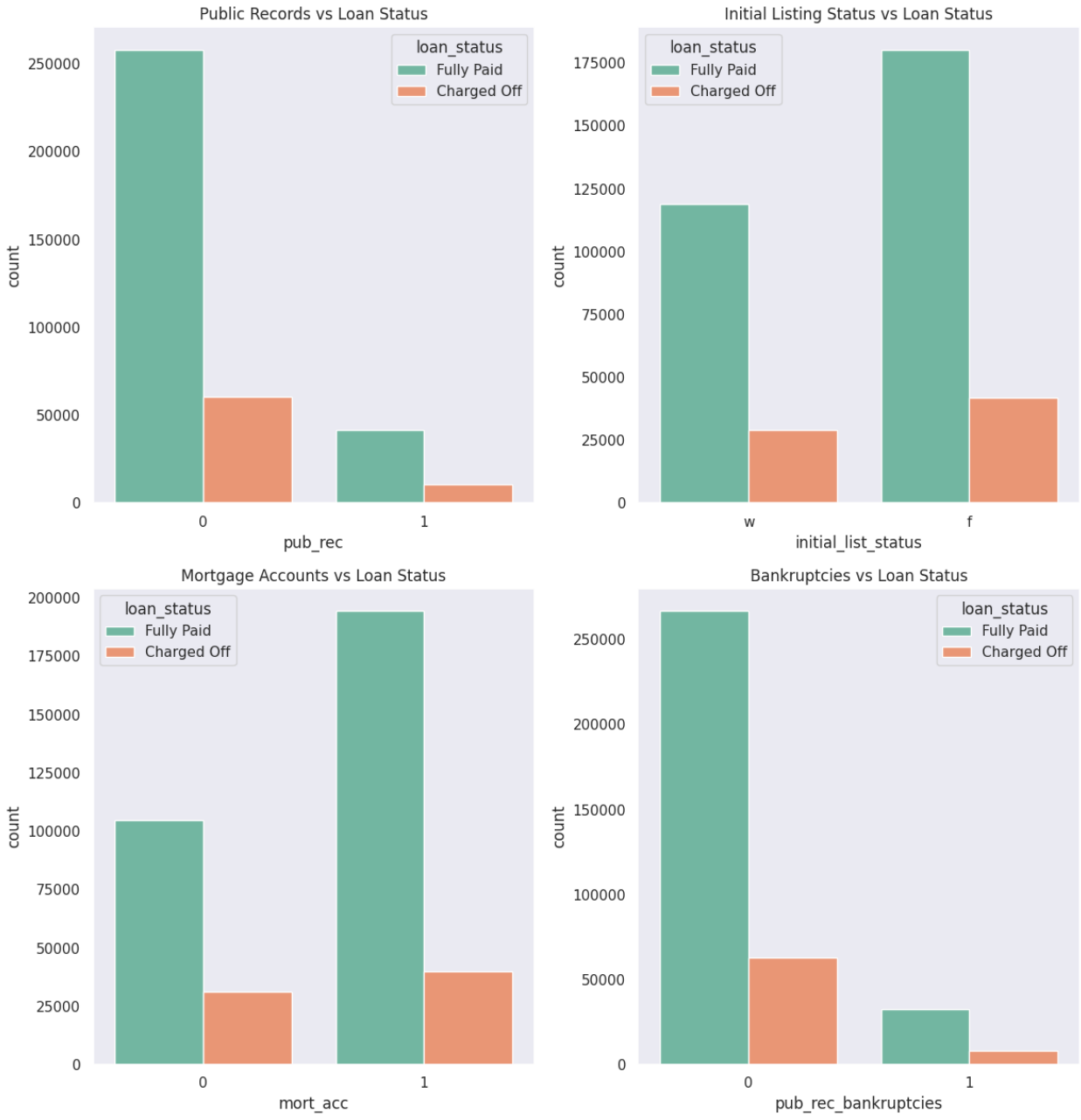
Public Records & Bankruptcies

pub_rec and pub_rec_bankruptcies are very strongly correlated (0.89), as bankruptcies are a type of public record. Including both in the model might cause multicollinearity.

Data Preprocessing using Feautre Engineering

```
# Binary transformation using lambda functions
df['pub_rec'] = df['pub_rec'].apply(lambda x: 1 if x > 0 else 0)
df['mort_acc'] = df['mort_acc'].apply(lambda x: 1 if x >= 1 else 0)
df['pub_rec_bankruptcies'] = df['pub_rec_bankruptcies'].apply(lambda
x: 1 if x >= 1 else 0)

# Set up the plot grid
plt.figure(figsize=(12, 24))

# Plot 1: Public Records vs Loan Status
plt.subplot(4, 2, 1)
sns.countplot(x='pub_rec', data=df, hue='loan_status', palette='Set2')
plt.title('Public Records vs Loan Status')

# Plot 2: Initial Listing Status vs Loan Status
plt.subplot(4, 2, 2)
sns.countplot(x='initial_list_status', data=df, hue='loan_status',
palette='Set2')
plt.title('Initial Listing Status vs Loan Status')

# Plot 3: Mortgage Accounts (Flagged) vs Loan Status
plt.subplot(4, 2, 3)
sns.countplot(x='mort_acc', data=df, hue='loan_status',
palette='Set2')
plt.title('Mortgage Accounts vs Loan Status')

# Plot 4: Bankruptcies (Flagged) vs Loan Status
plt.subplot(4, 2, 4)
sns.countplot(x='pub_rec_bankruptcies', data=df, hue='loan_status',
palette='Set2')
plt.title('Bankruptcies vs Loan Status')
```

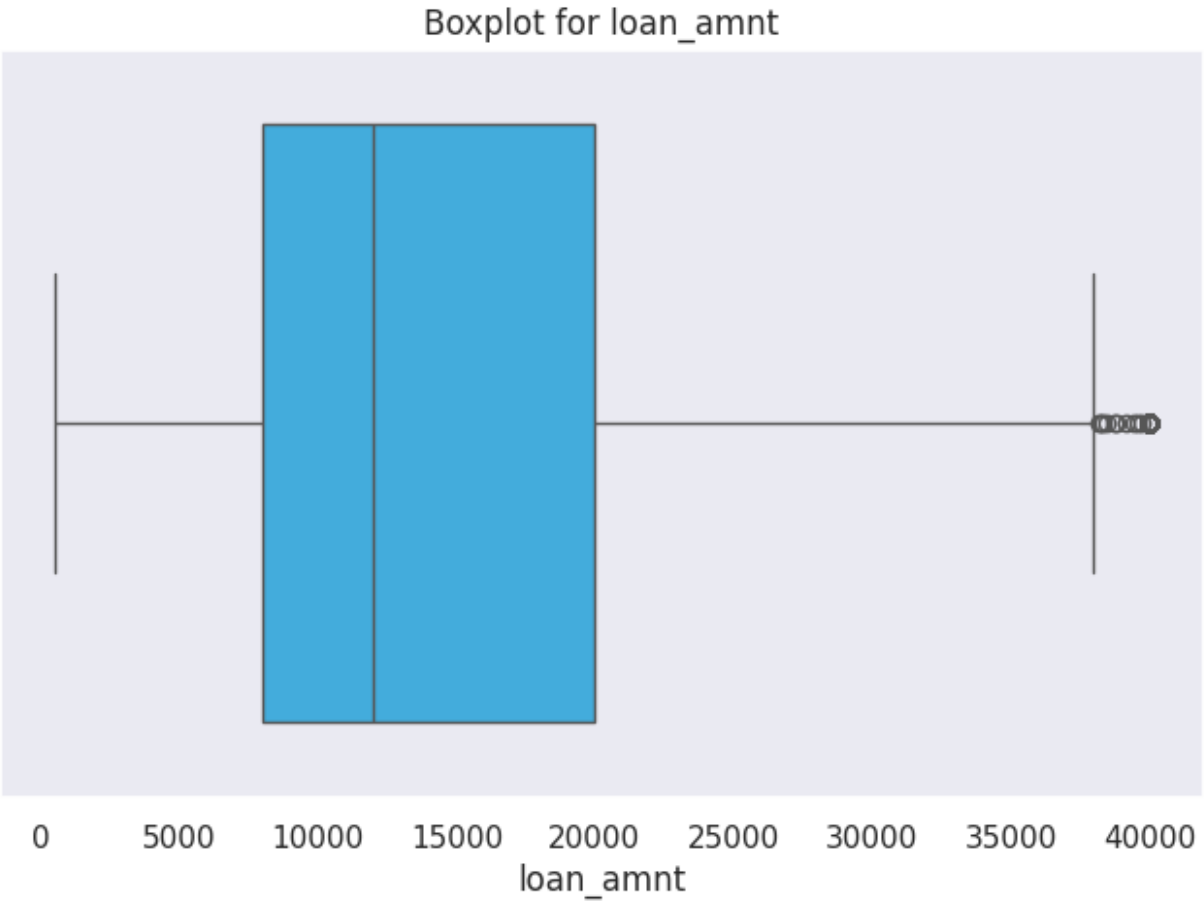
```
plt.tight_layout()
plt.show()
```

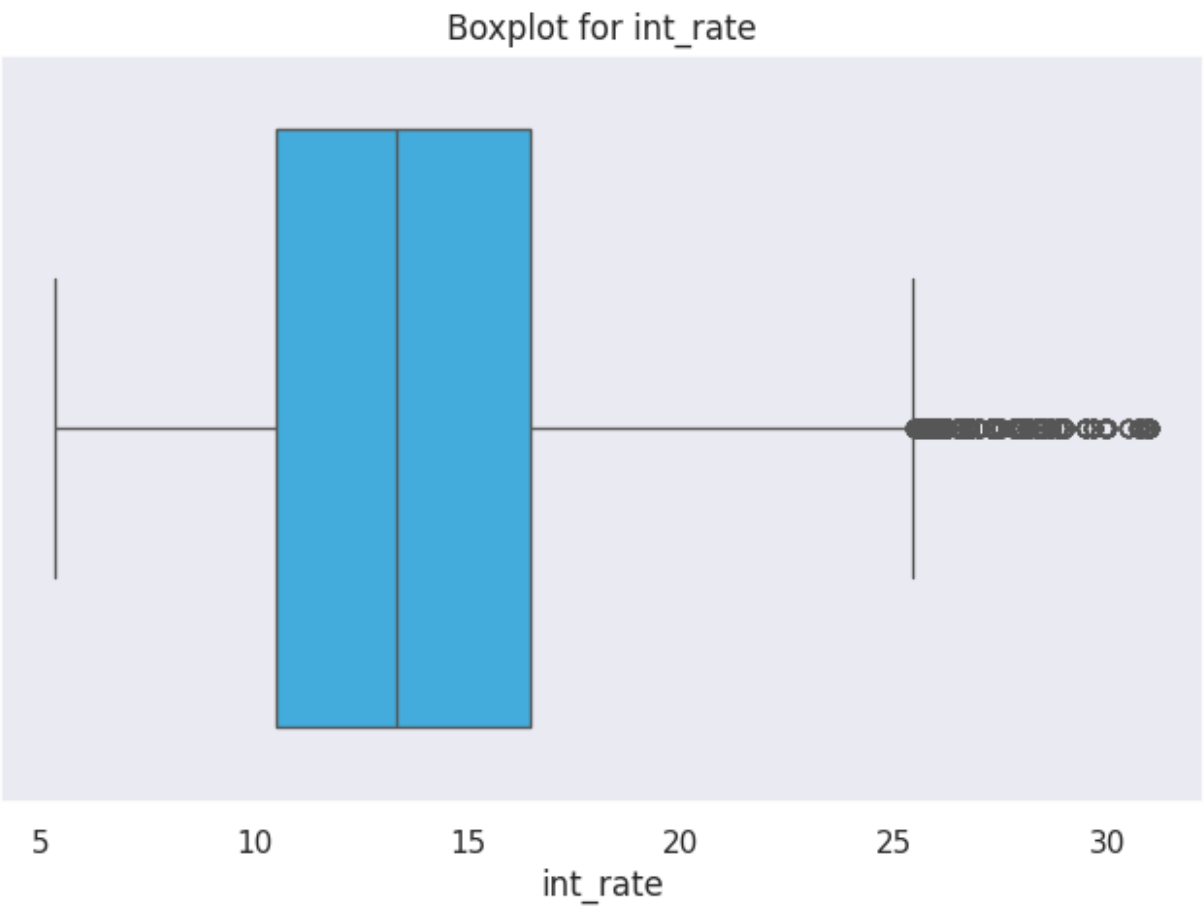


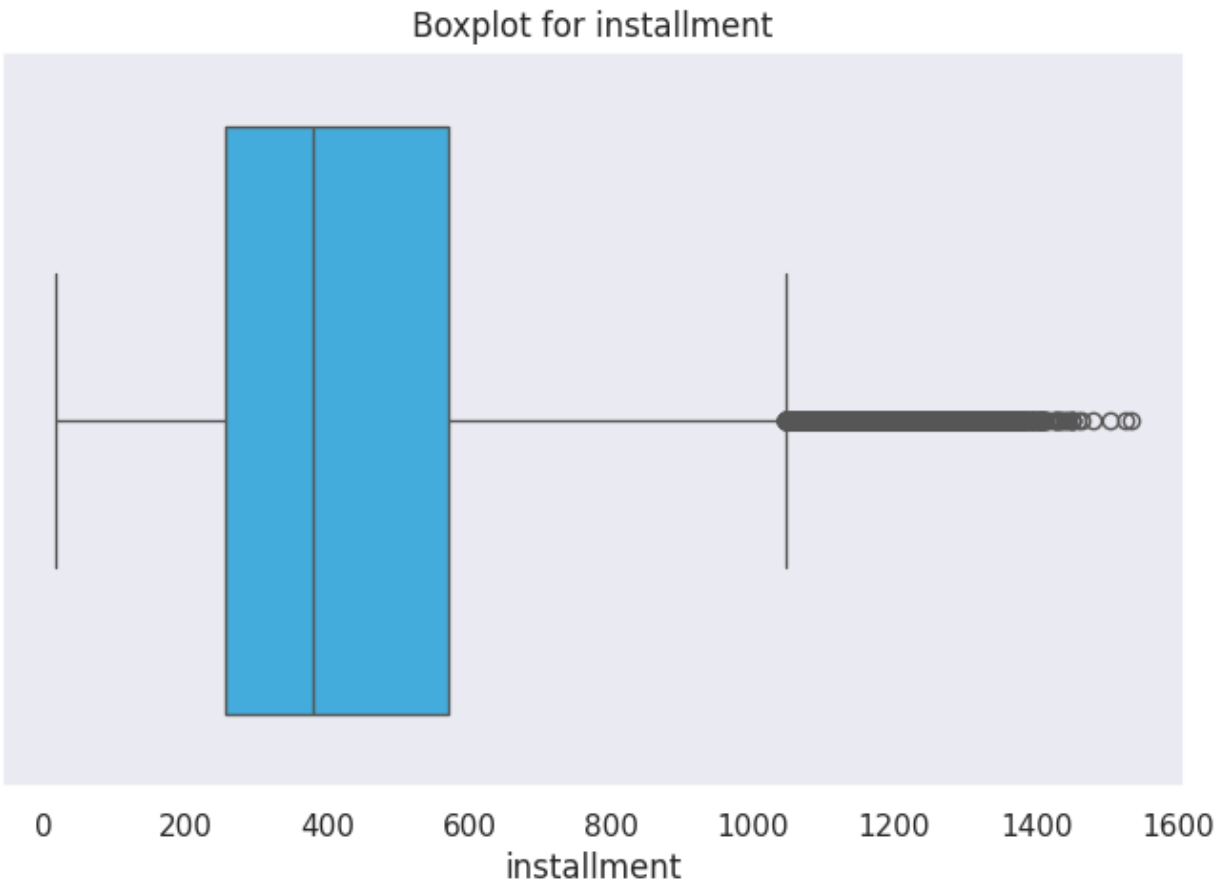
Outlier Detection

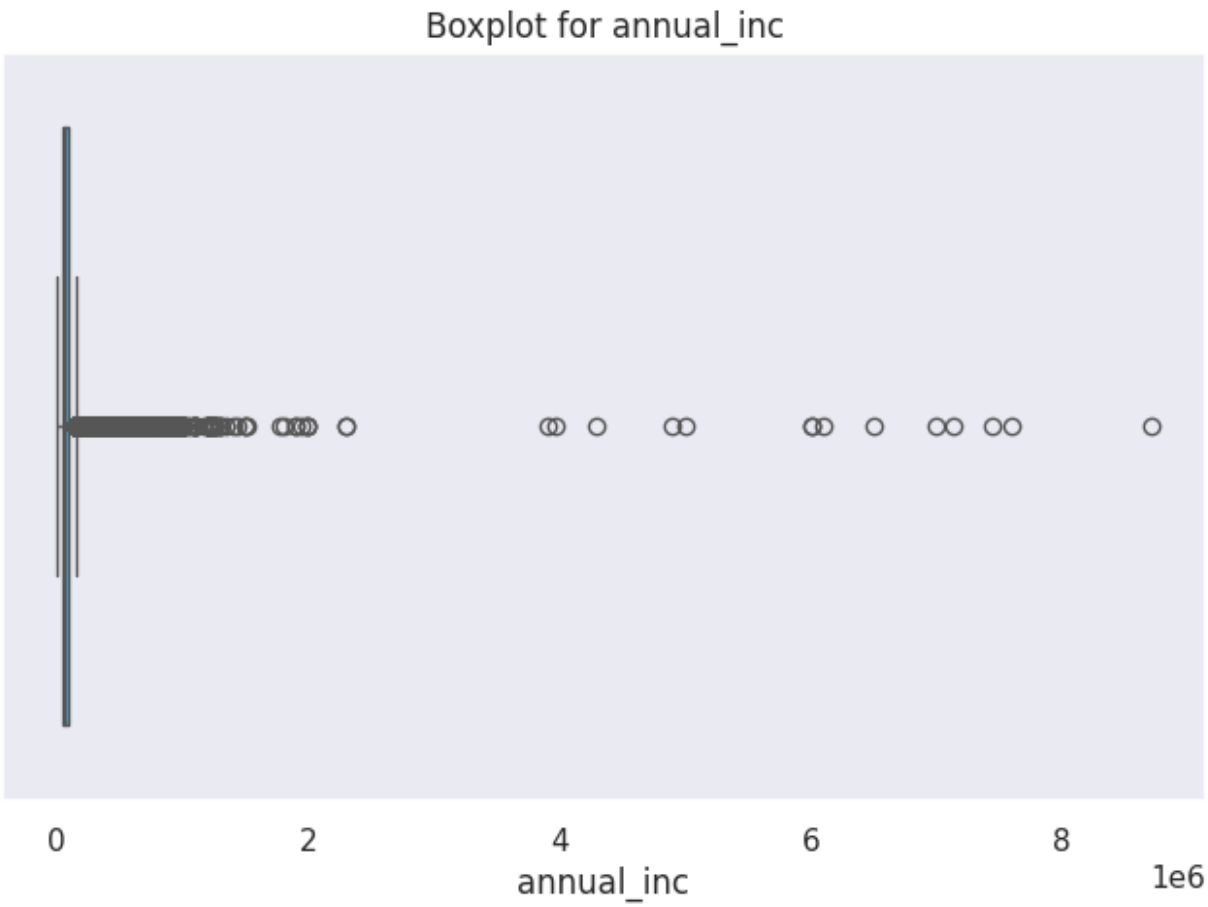
```
def box_plot(col):
    if col in df.columns:
        plt.figure(figsize=(8, 5))
        sns.boxplot(x=df[col],color="#29B6F6")
        plt.title('Boxplot for {}'.format(col))
```

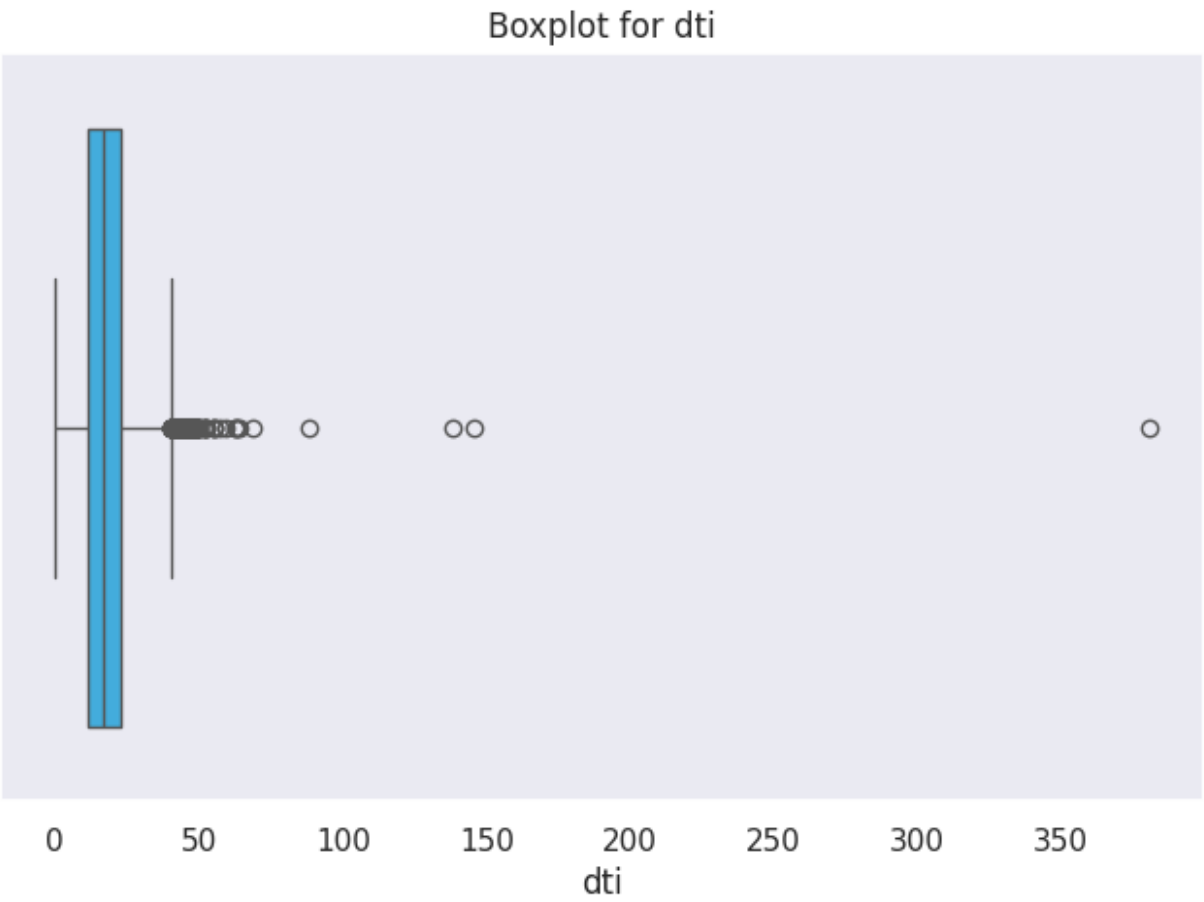
```
        plt.show()
    else:
        print(f"Column '{col}' not found in the DataFrame.")

for col in n_columns:
    box_plot(col)
```

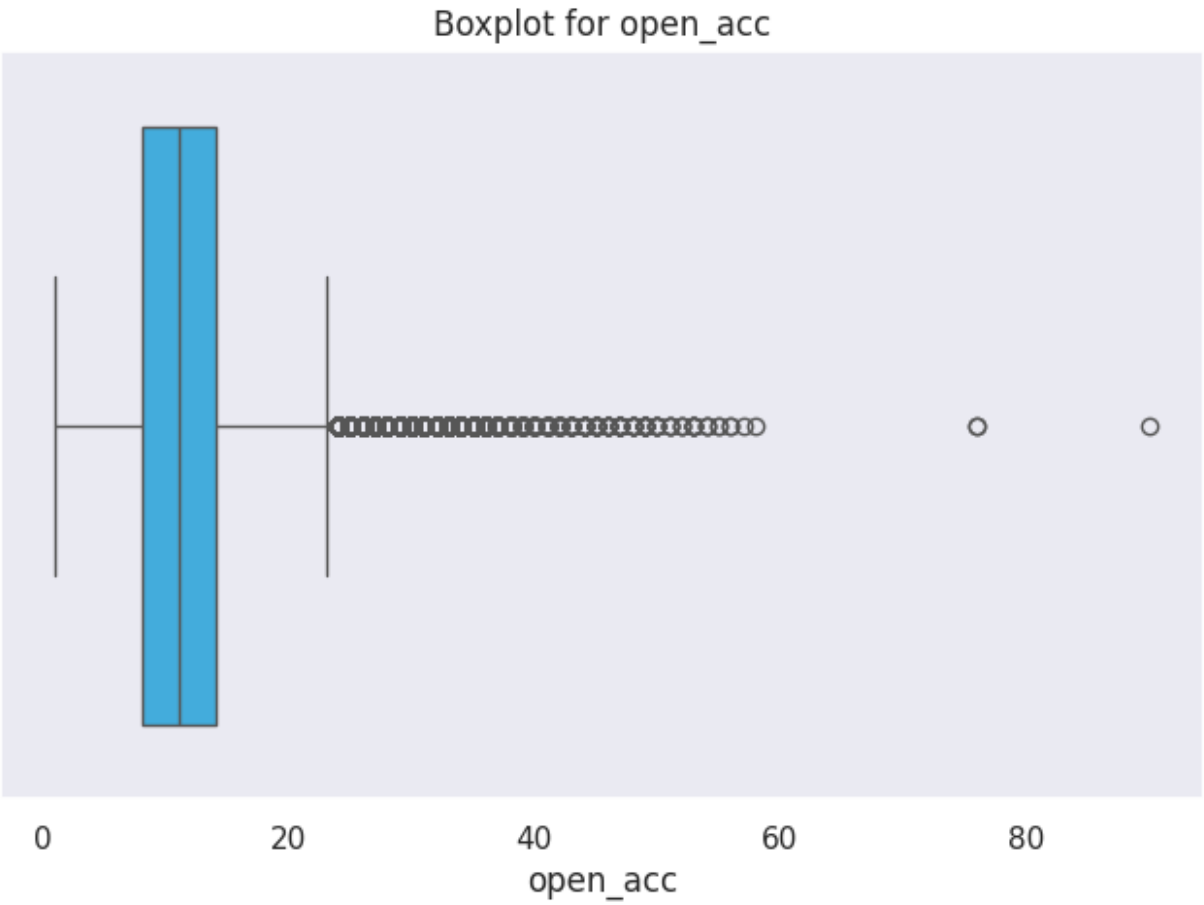


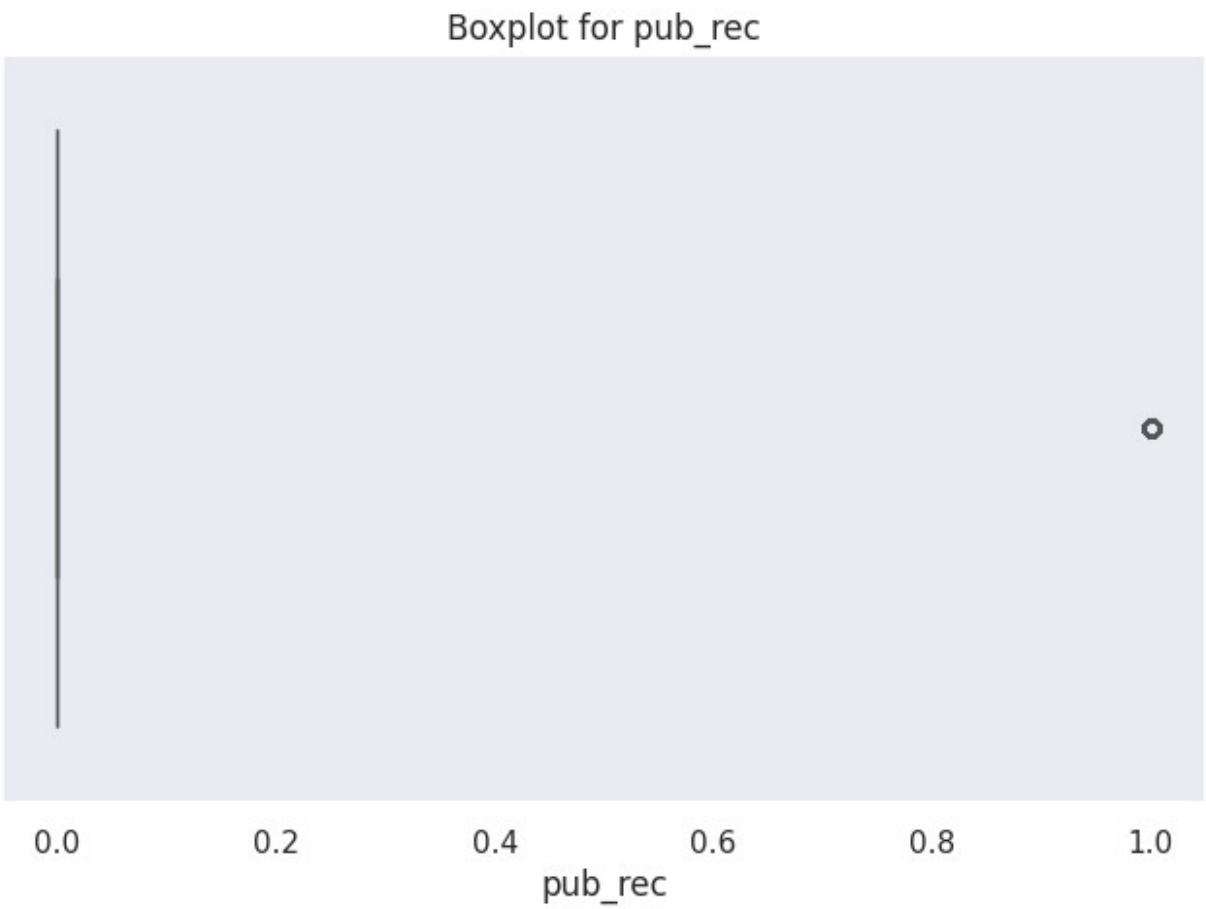

Boxplot for int_rate
5
10
15
20
25
30
int_rate

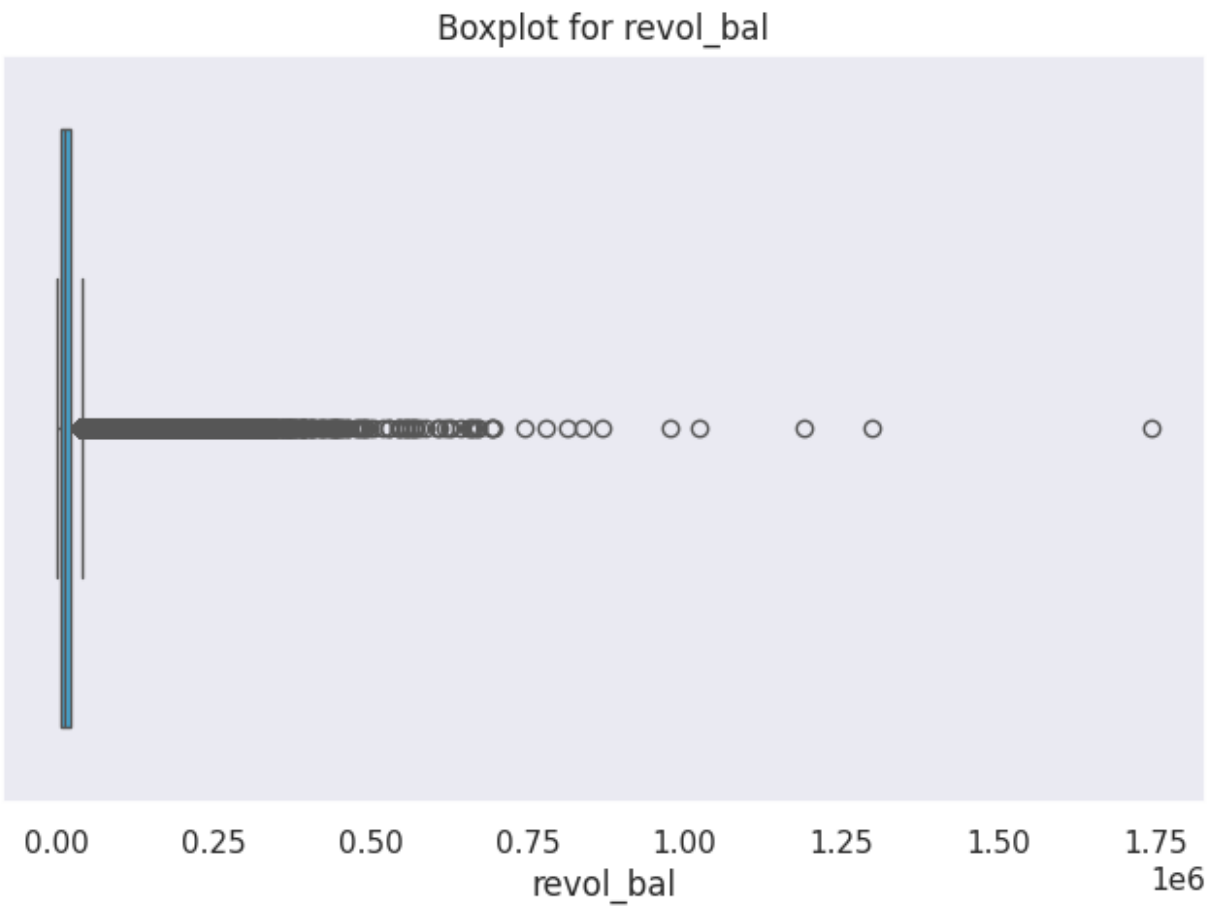

Boxplot for installment
0
200
400
600
800
1000
1200
1400
1600
installment

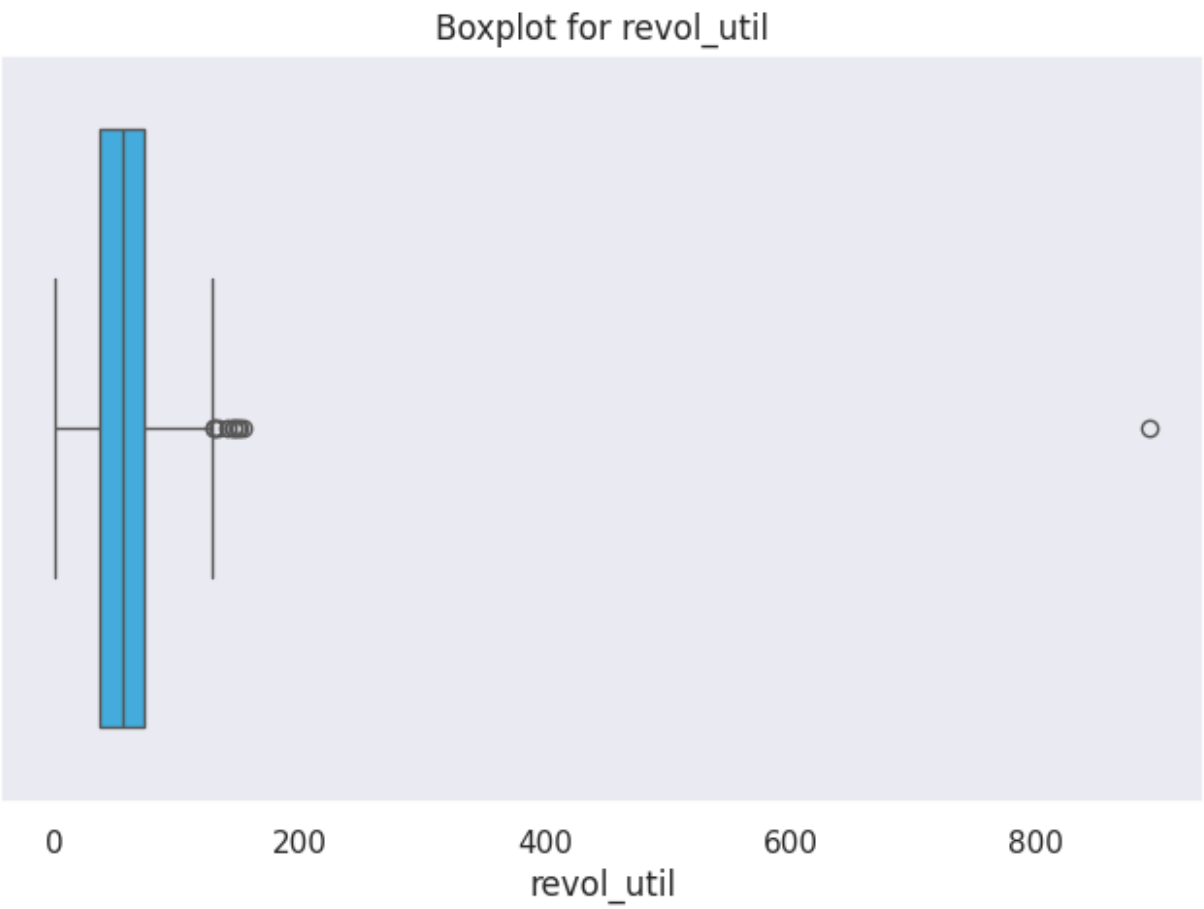

Boxplot for annual_inc
0
2
4
6
8
annual_inc
1e6

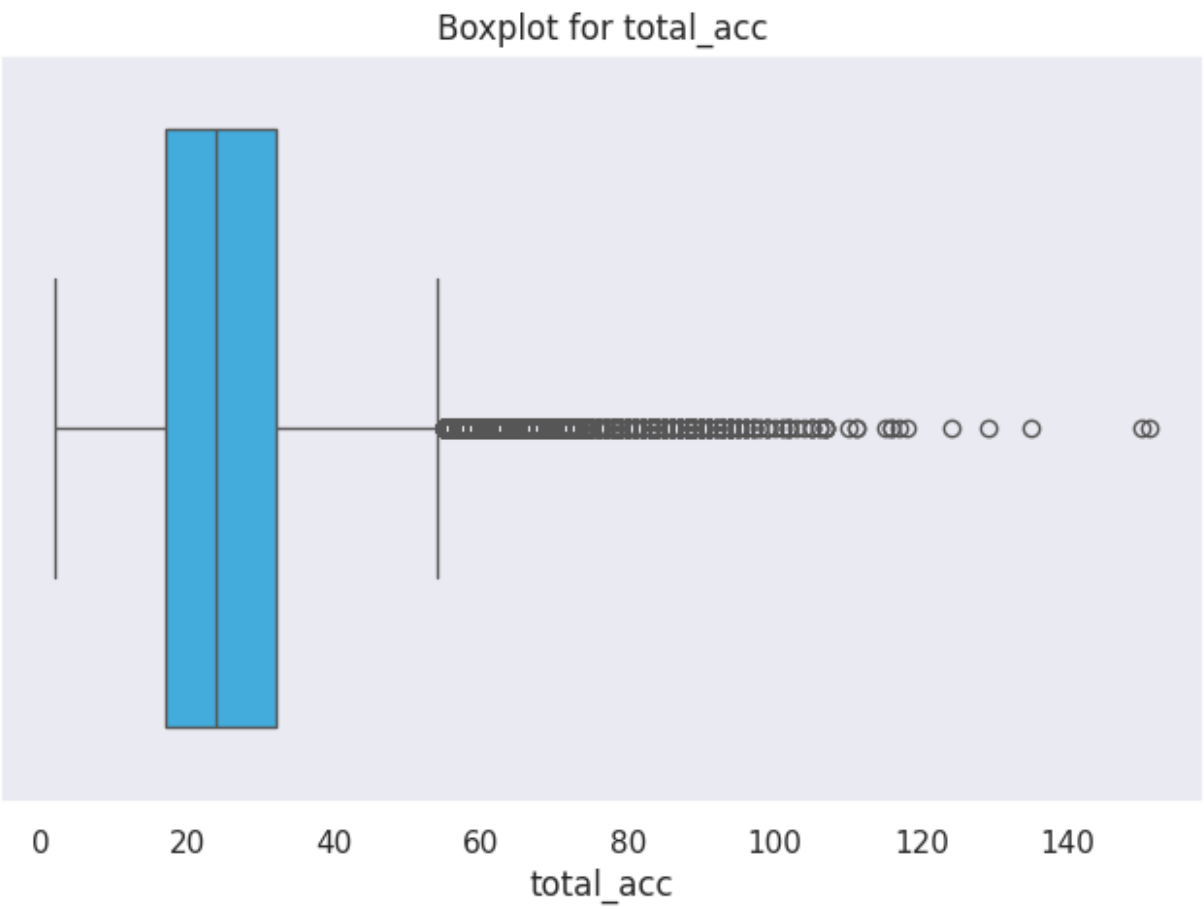

Boxplot for dti
0
50
100
150
200
250
300
350
dti

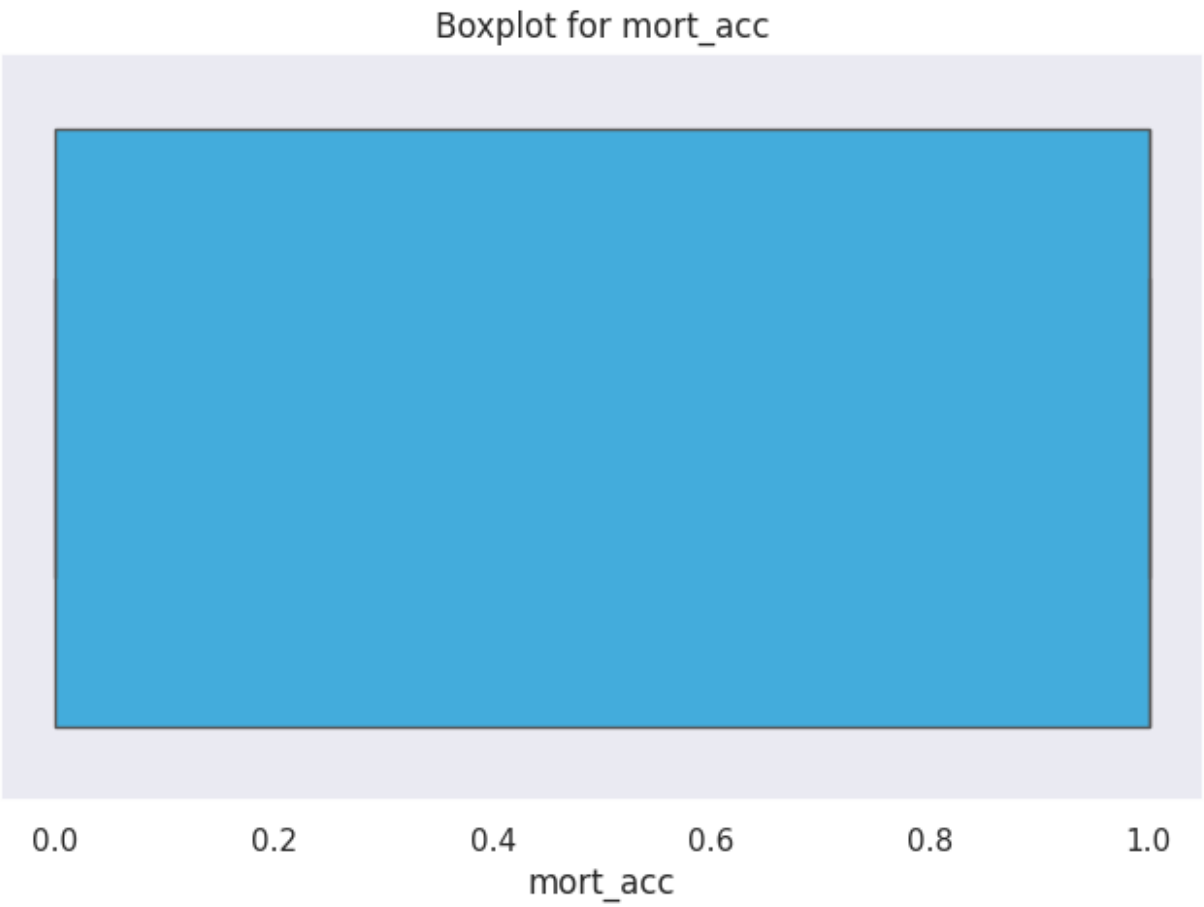

Boxplot for open_acc
0
20
40
60
80
open_acc

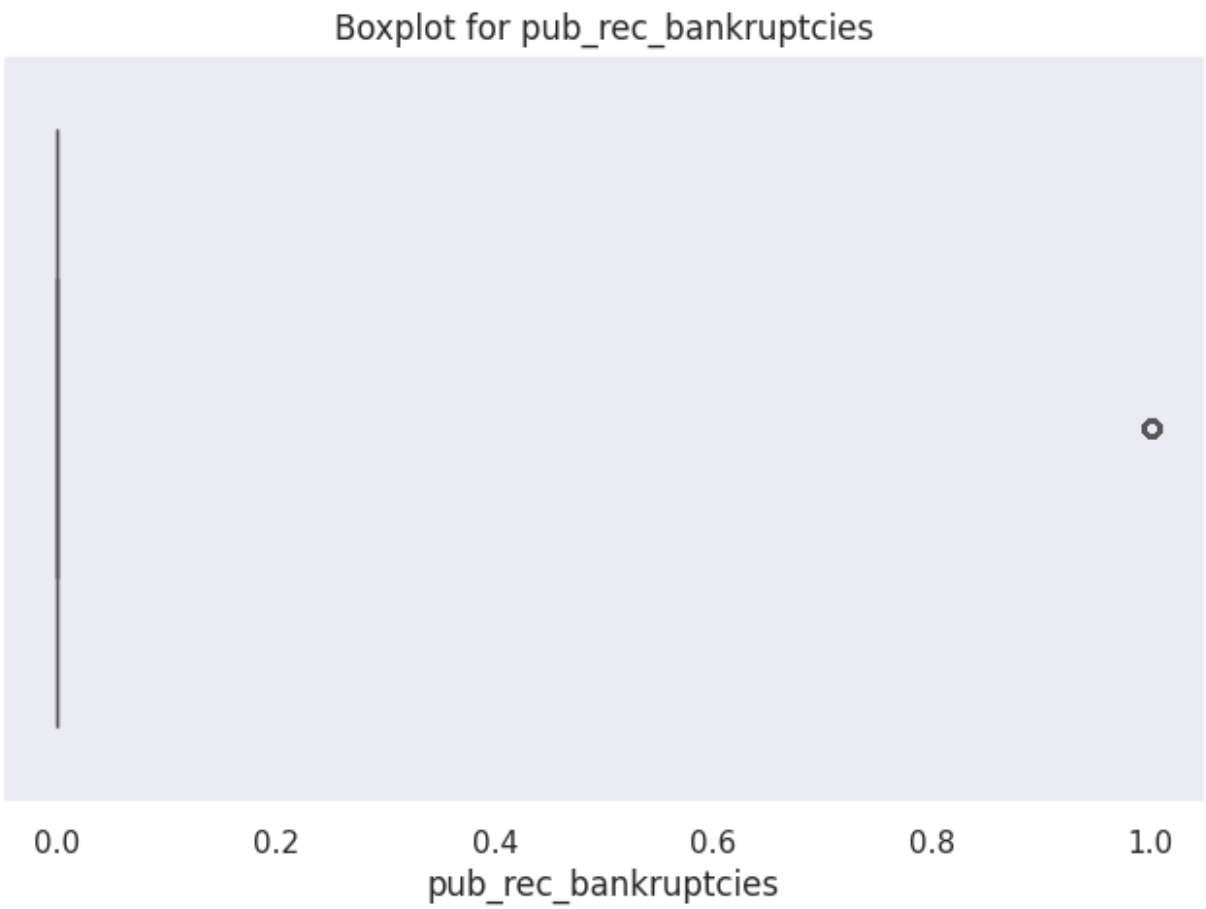

Boxplot for pub_rec
0.0
0.2
0.4
0.6
0.8
1.0
pub_rec


Boxplot for revol_bal
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
1e6
revol_bal


Boxplot for revol_util
0
200
400
600
800
revol_util

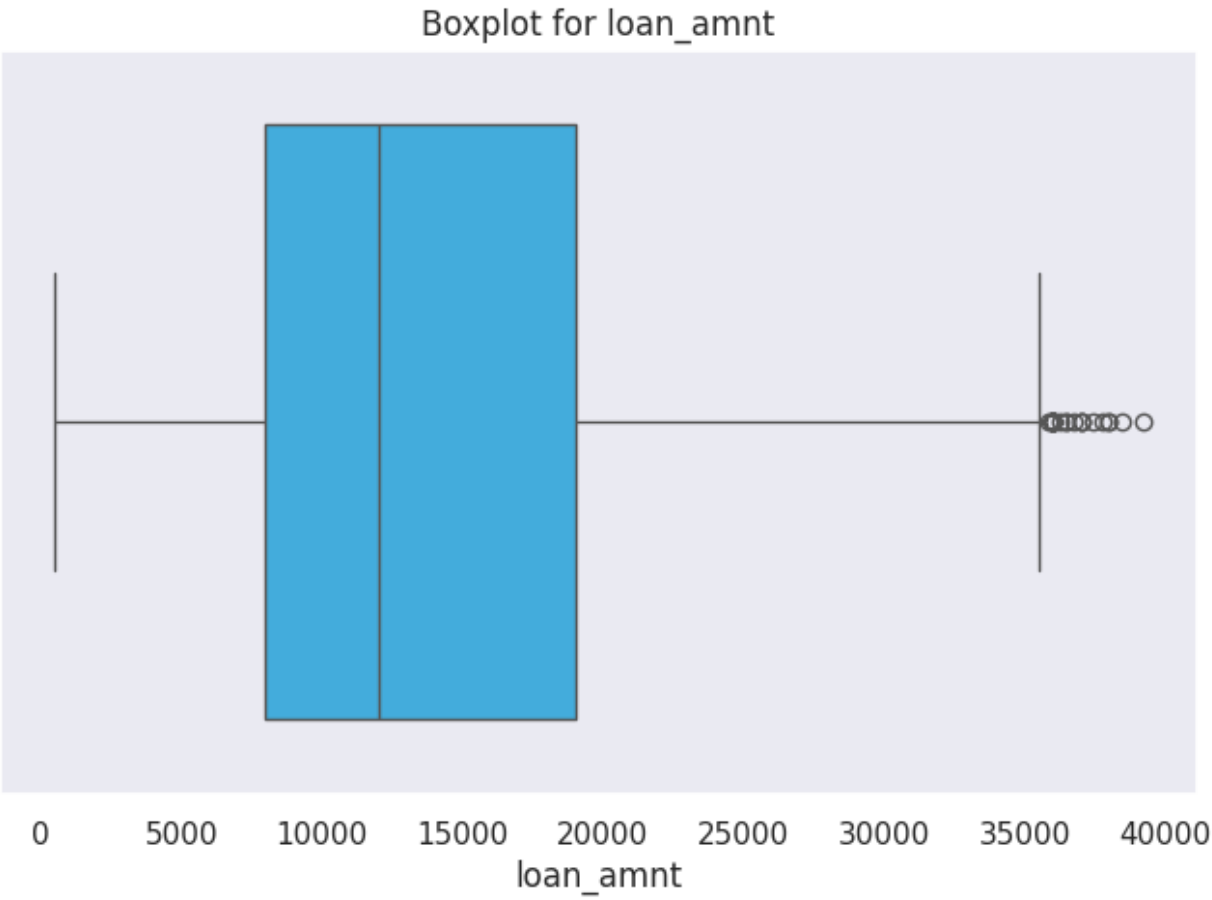

Boxplot for total_acc
0
20
40
60
80
100
120
140
total_acc

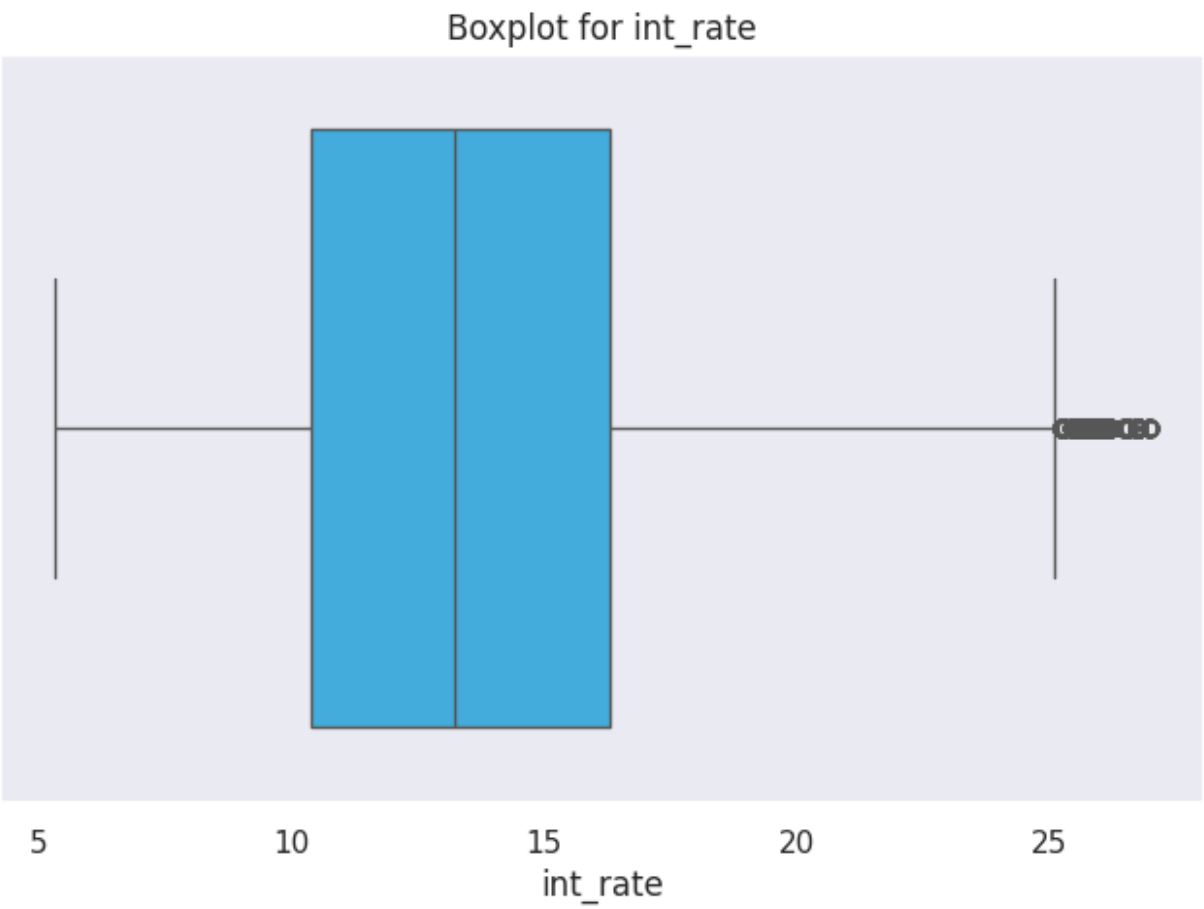

Boxplot for mort_acc
0.0
0.2
0.4
0.6
0.8
1.0
mort_acc

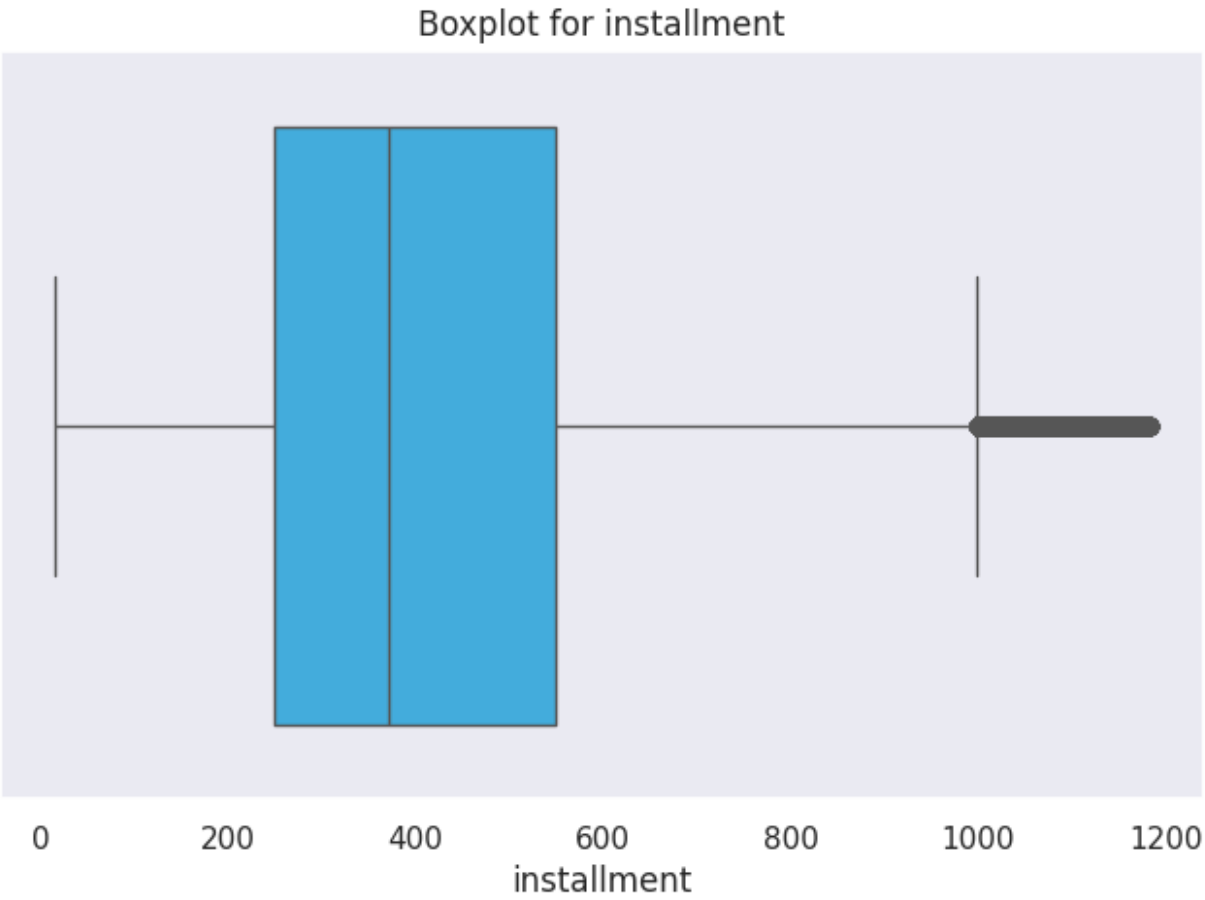


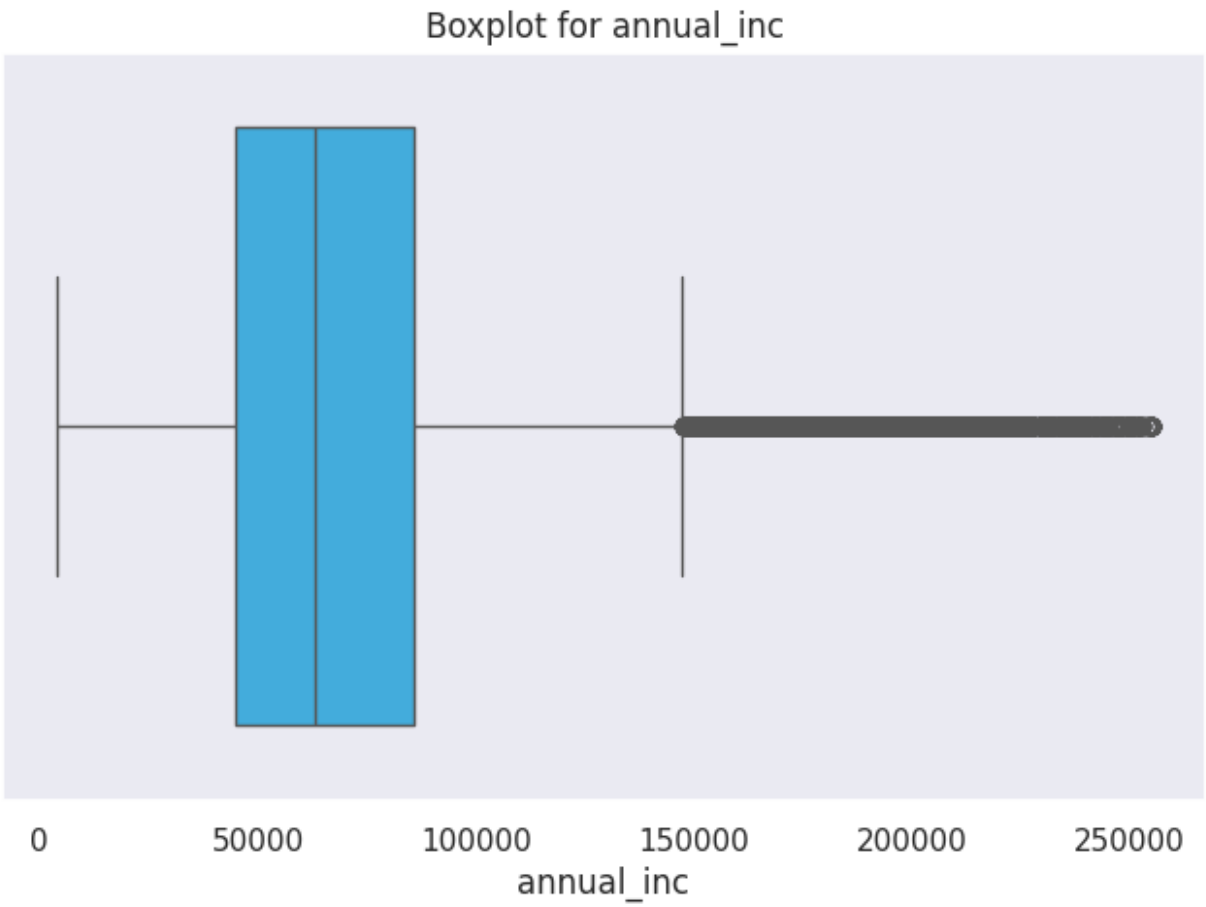
Outlier treatment

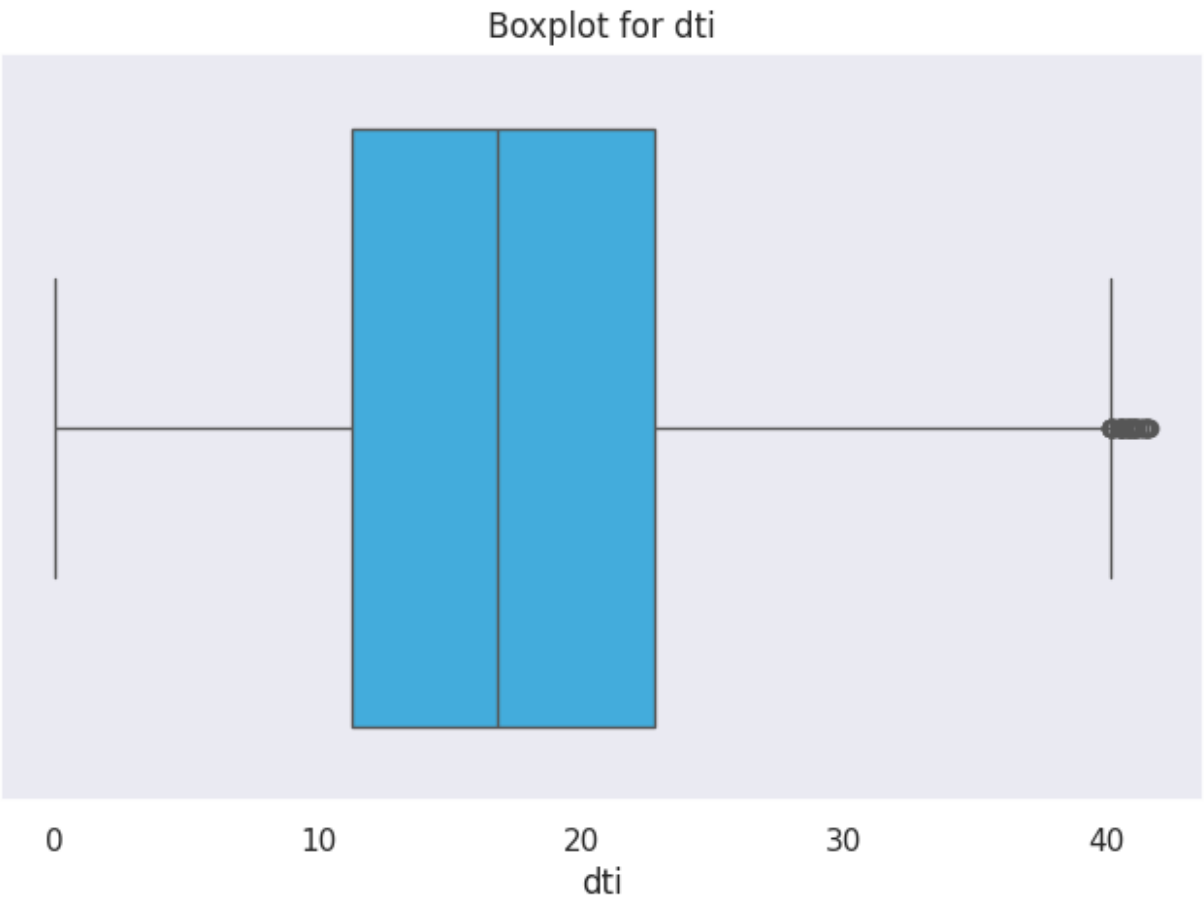
```
for col in n_columns:
    if col in df.columns:
        mean = df[col].mean()
        std = df[col].std()
        upper_limit = mean + 3 * std
        lower_limit = mean - 3 * std
        df = df[(df[col] < upper_limit) & (df[col] > lower_limit)]

def box_plot(col):
    if col in df.columns:
        plt.figure(figsize=(8, 5))
        sns.boxplot(x=df[col],color="#29B6F6")
        plt.title('Boxplot for {}'.format(col))
        plt.show()
    else:
        print(f"Column '{col}' not found in the DataFrame.")

for col in n_columns:
    box_plot(col)
```

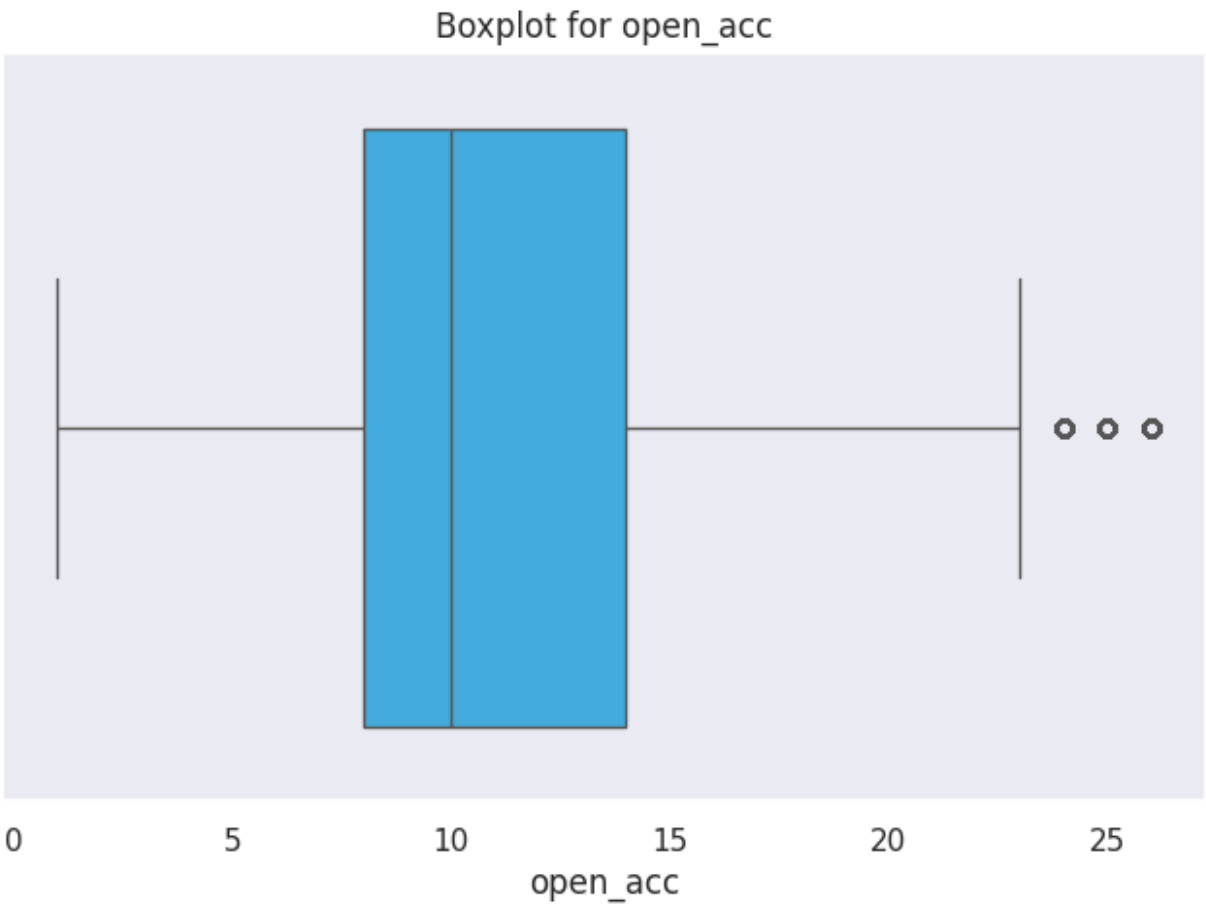

Boxplot for loan_amnt
0
5000
10000
15000
20000
25000
30000
35000
40000
loan_amnt

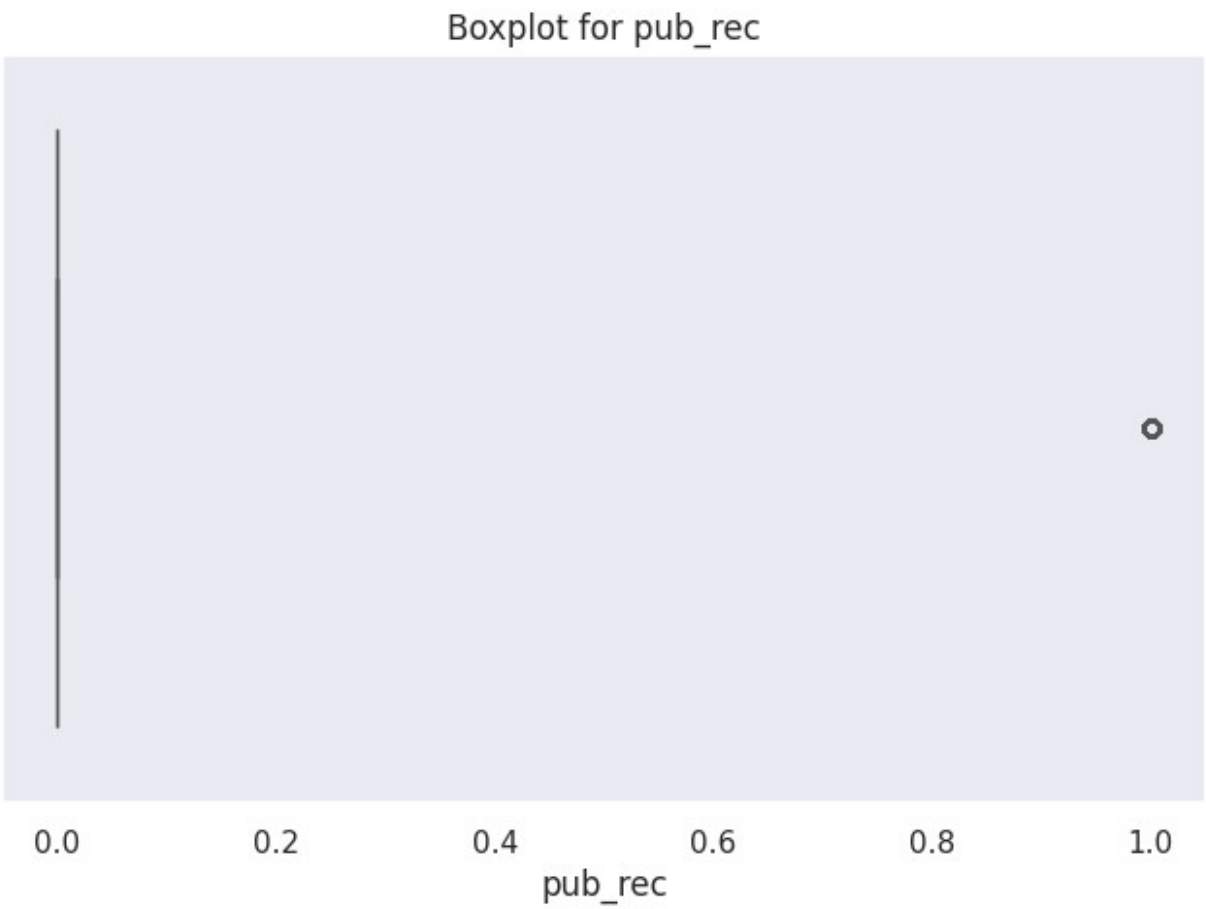

Boxplot for int_rate
5
10
15
20
25
int_rate

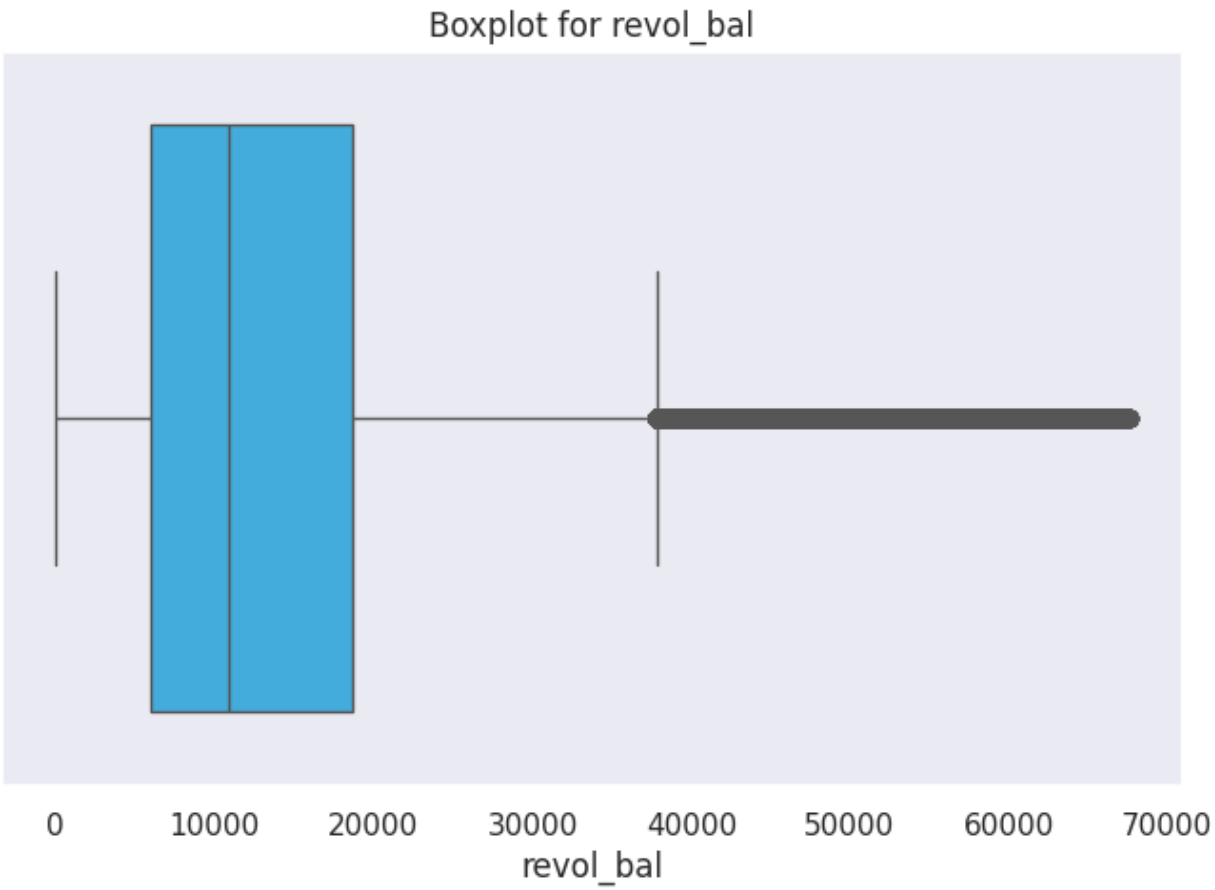

Boxplot for installment
0
200
400
600
800
1000
1200
installment

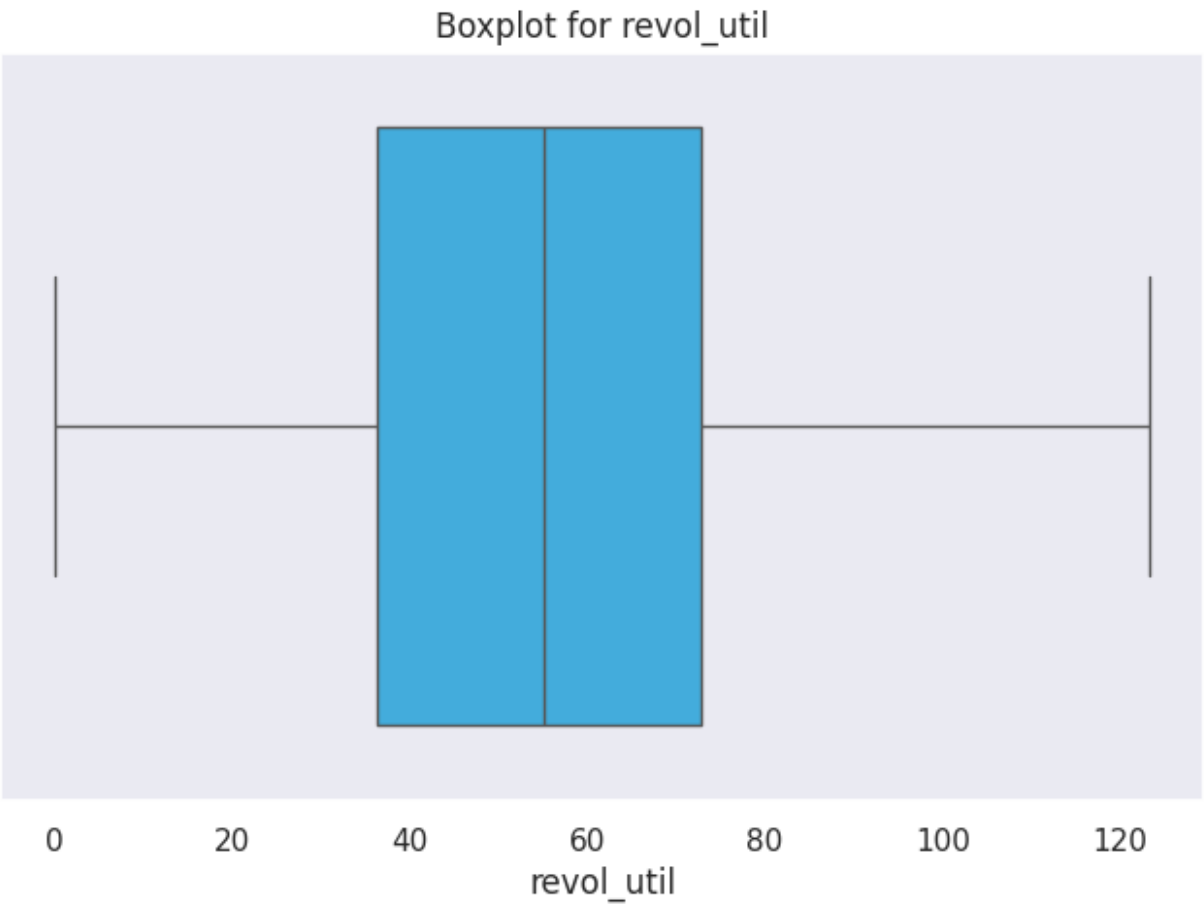

Boxplot for annual_inc
0
50000
100000
150000
200000
250000
annual_inc

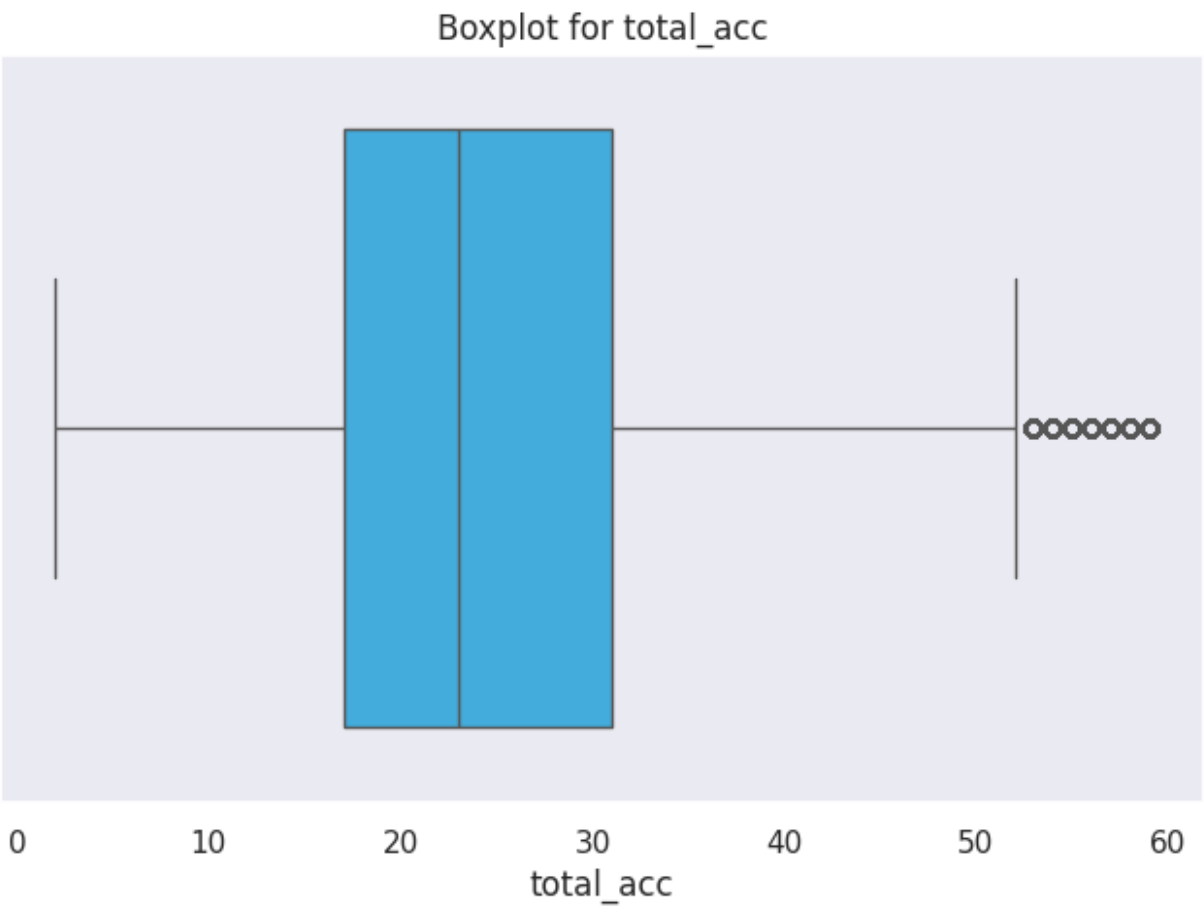

Boxplot for dti
0
10
20
30
40
dti

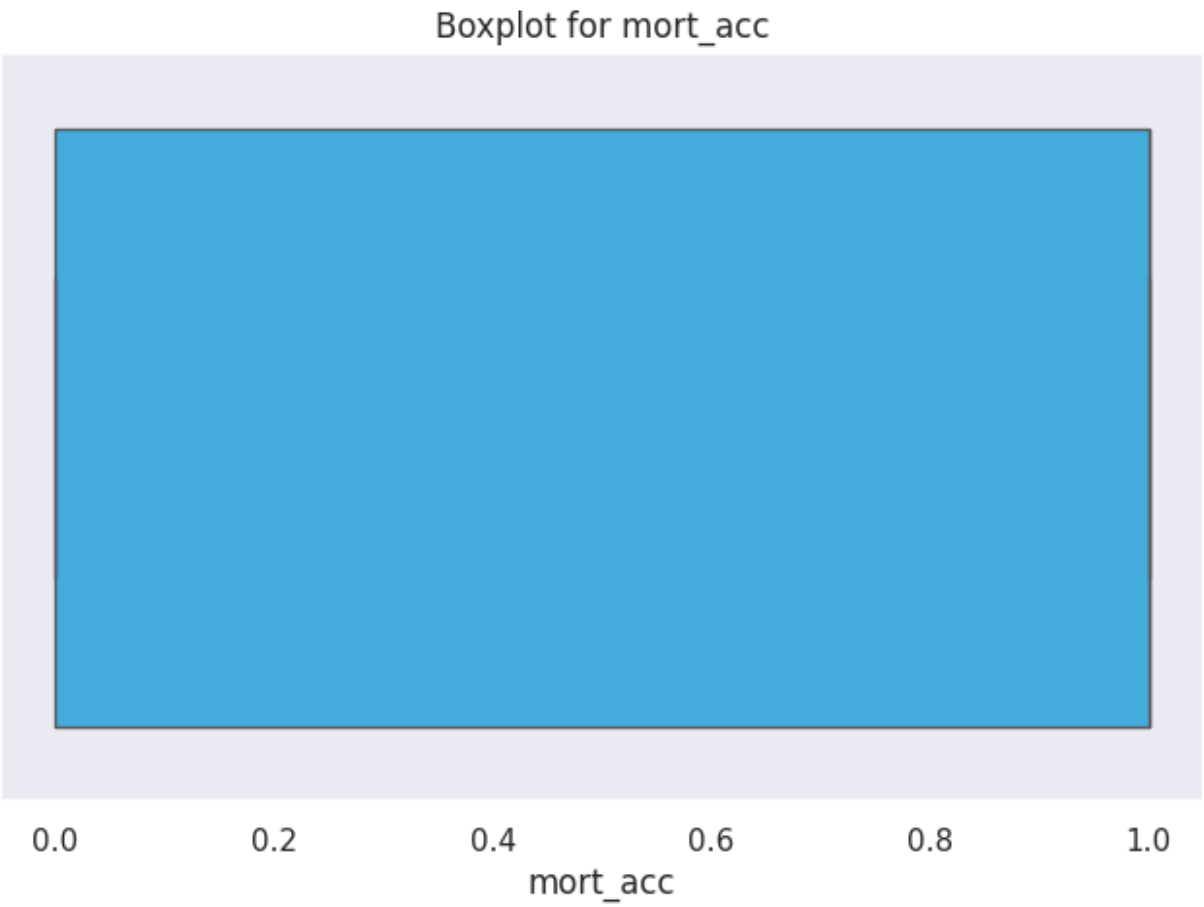

Boxplot for open_acc
0
5
10
15
20
25
open_acc

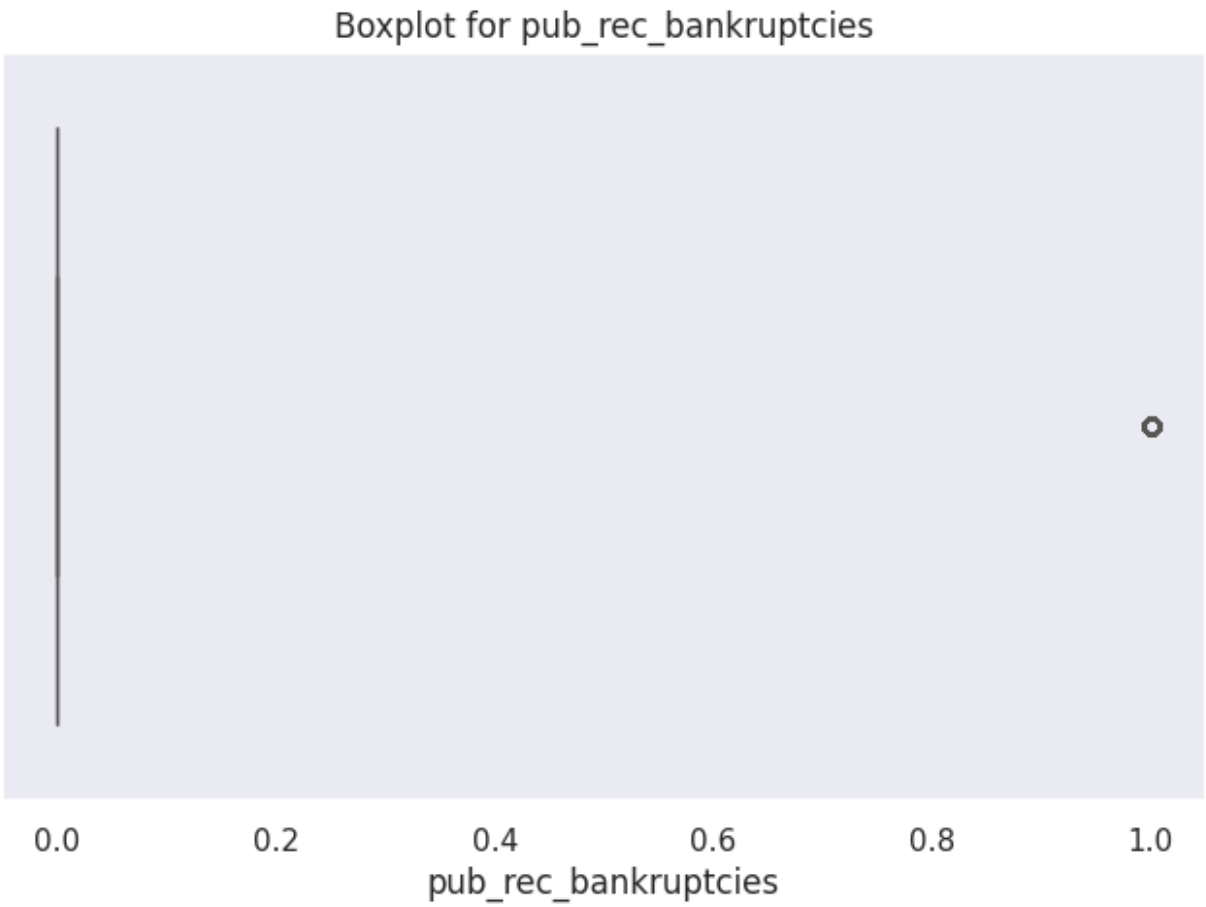

Boxplot for pub_rec
0.0
0.2
0.4
0.6
0.8
1.0
pub_rec


Boxplot for revol_bal
0
10000
20000
30000
40000
50000
60000
70000
revol_bal


Boxplot for revol_util
0
20
40
60
80
100
120
revol_util


Boxplot for total_acc
0
10
20
30
40
50
60
total_acc


Boxplot for mort_acc
0.0
0.2
0.4
0.6
0.8
1.0
mort_acc




```
# Define a mapping for the 'term' column to convert string values to
numeric
term_values = {' 36 months': 36, ' 60 months': 60}
# Apply the mapping to the 'term' column, replacing string values with
corresponding integers
df['term'] = df['term'].map(term_values)

# Map 'loan_status' to binary values: 'Fully Paid' becomes 0, 'Charged
Off' becomes 1
df['loan_status'] = df['loan_status'].map({'Fully Paid': 0, 'Charged
Off': 1})

# Define a mapping for 'initial_list_status' where 'w' is mapped to 0
and 'f' is mapped to 1
list_status = {'w': 0, 'f': 1}
# Apply the mapping to the 'initial_list_status' column
df['initial_list_status'] = df['initial_list_status'].map(list_status)

# Create a new 'zip_code' column by extracting the last 5 characters
from the 'address' column
# This assumes the last 5 characters of 'address' represent the ZIP
code
```

```
df['zip_code'] = df['address'].apply(lambda x: x[-5:])

# Calculate the percentage distribution of different 'zip_code' values
and display them as a percentage
df['zip_code'].value_counts(normalize=True) * 100
```

```
zip_code
70466    14.375337
30723    14.289710
22690    14.272299
48052    14.127019
00813    11.605591
29597    11.548792
05113    11.519108
93700     2.768605
11650     2.762896
86630     2.730643
Name: proportion, dtype: float64
```

Dropping low priority columns

```
# Dropping of unnecessary columns

unnecessary_columns=['issue_d', 'emp_title', 'title',
'sub_grade','address', 'earliest_cr_line', 'emp_length']

df.drop(unnecessary_columns,axis=1, inplace=True)
```

One hot encoding

```
# List of categorical columns that we want to one-hot encode
dummies = ['purpose', 'zip_code', 'grade', 'verification_status',
'application_type', 'home_ownership']

# Apply one-hot encoding to the specified columns, and drop the first
category for each (to avoid multicollinearity)
data = pd.get_dummies(df, columns=dummies, drop_first=True)

# Set display options to show all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Set display options to show all rows in the DataFrame output
pd.set_option('display.max_rows', None)

X=data.drop('loan_status',axis=1)
y=data['loan_status']
X_train, X_test, y_train, y_test
=train_test_split(X,y,test_size=0.30,stratify=y,random_state=42)
print(X_train.shape)
print(X_test.shape)
```

```
(245249, 51)
(105108, 51)
```

Model Building

```
scaler = MinMaxScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

logreg=LogisticRegression(max_iter=1000)

logreg.fit(X_train,y_train)
```

```
LogisticRegression(max_iter=1000)
```

```
y_pred = logreg.predict(X_test)

print('Accuracy of Logistic Regression Classifier on test set:
{:.3f}'.format(logreg.score(X_test, y_test)))
```

```
Accuracy of Logistic Regression Classifier on test set: 0.891
```

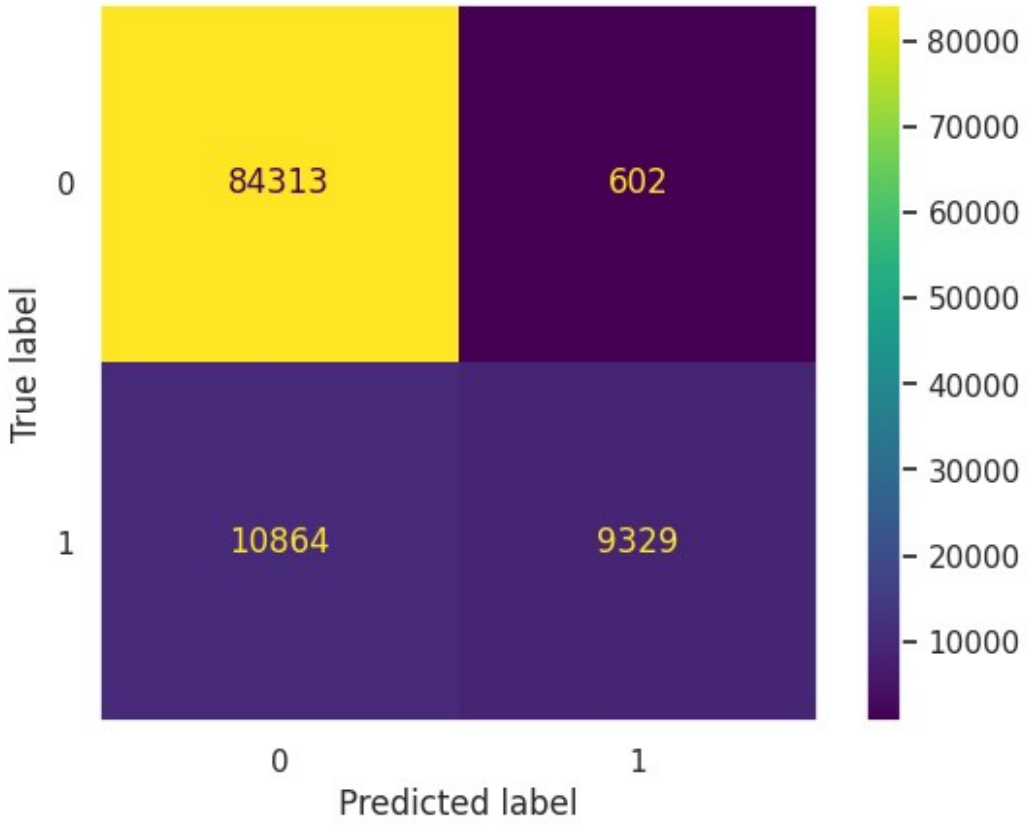
```
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.89      0.99      0.94     84915
           1       0.94      0.46      0.62     20193

    accuracy                           0.89    105108
   macro avg       0.91      0.73      0.78    105108
weighted avg       0.90      0.89      0.88    105108
```

```
#Plot confusion Matrix

confusion_matrix=confusion_matrix(y_test,y_pred)

print(confusion_matrix)

ConfusionMatrixDisplay(confusion_matrix=confusion_matrix,
display_labels=logreg.classes_).plot()
```

```
[[84313   602]
 [10864  9329]]
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x7dde9e33de10>
```



ROC Curve (Receiver Operating Characteristic Curve)

The ROC curve is a graphical representation used to evaluate the performance of a classification model across all possible classification thresholds. It plots two key metrics:

True Positive Rate (TPR): Also known as Recall, it measures the proportion of actual positives correctly identified by the model. Mathematically, it's defined as:

TPR=(TP)/(TP+FN) where TP = True Positives, FN = False Negatives.

False Positive Rate (FPR): This metric shows the proportion of actual negatives that are incorrectly classified as positive. It is defined as:

FPR=(FP)/(FP+TN) where FP = False Positives, TN = True Negatives.

The ROC curve plots TPR on the Y-axis and FPR on the X-axis for different thresholds. As the threshold is lowered, more instances are classified as positive, which increases both True Positives and False Positives. The curve helps visualize how well the model distinguishes between positive and negative cases.

AUC (Area Under the ROC Curve) AUC stands for the Area Under the ROC Curve, and it measures the overall performance of the classification model. It represents the area under the ROC curve, ranging from 0 to 1.

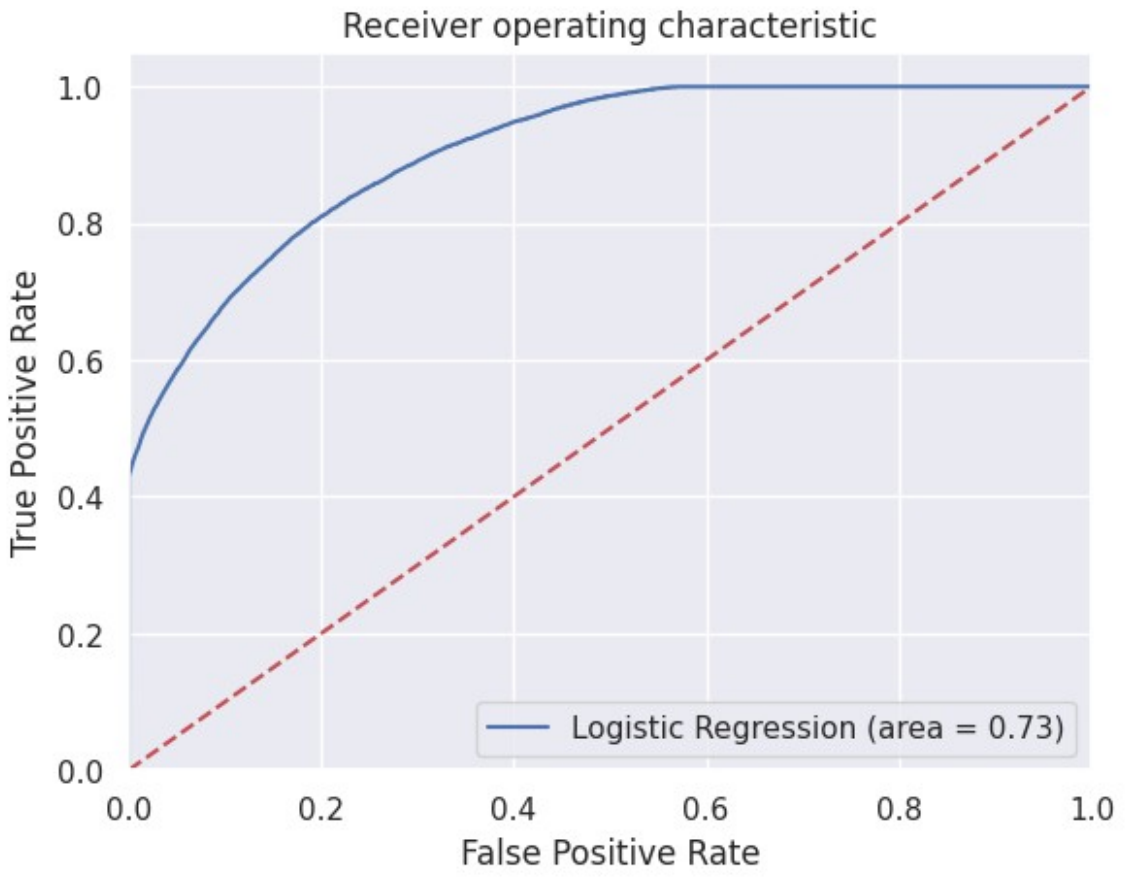
An AUC of 1 indicates perfect model performance, meaning it ranks all positive instances higher than negative ones.

An AUC of 0.5 suggests that the model is no better than random chance.

Interpreting AUC:

AUC can be seen as the probability that the model will correctly rank a randomly chosen positive instance higher than a randomly chosen negative one. A higher AUC means better performance in distinguishing between positive and negative examples across all thresholds.

```
logit_roc_auc = roc_auc_score(y_test, logreg.predict(X_test))

fpr, tpr, thresholds = roc_curve(y_test, logreg.predict_proba(X_test)
[:,1])


plt.figure()
plt.plot(fpr, tpr, label='Logistic Regression (area = %0.2f)' %
logit_roc_auc)
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.grid()
plt.show()
```



Insights:

The ROC-AUC score is approximately 0.73, indicating that the model demonstrates a decent ability to distinguish between the positive and negative classes.

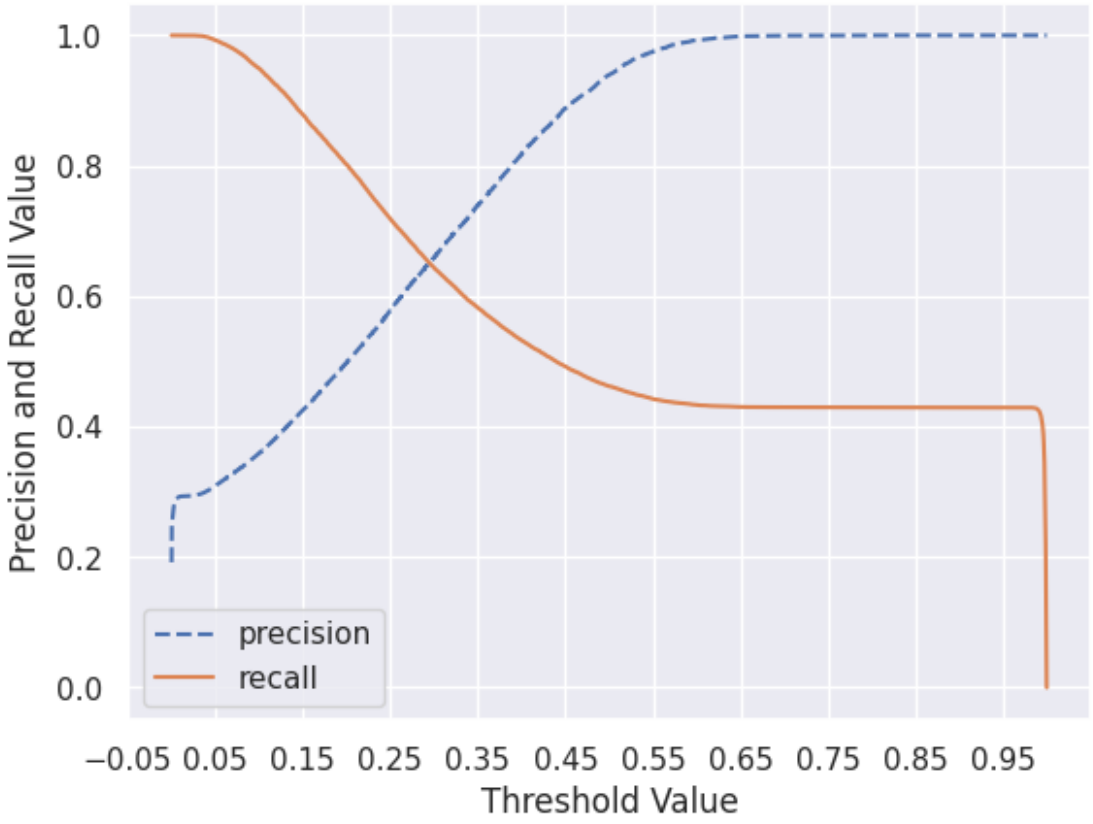
While the model is performing reasonably well, there is potential for further improvement.

Enhancing model performance could be achieved by collecting additional data, exploring more sophisticated models, or fine-tuning the hyperparameters.

Precision-Recall Curve

```
precisions, recalls, thresholds = precision_recall_curve(y_test,
logreg.predict_proba(X_test)[:, 1])

threshold_boundary = thresholds.shape[0]

# Plot precision
plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--',
label='precision')

# Plot recall
plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')
```

```
start, end = plt.xlim()
plt.xticks(np.round(np.arange(start, end, 0.1), 2))

plt.xlabel('Threshold Value')
plt.ylabel('Precision and Recall Value')
plt.legend()
plt.grid()
plt.show()
```



Insights:

The Precision score peaks at a threshold of 0.55, indicating that the model is highly accurate when predicting charged-off loans. This is crucial for businesses as it helps minimize false positives, enabling more confident and stable decision-making.

The Recall score is higher at lower thresholds but stabilizes beyond 0.55. This suggests that the model effectively identifies most of the actual defaulters early on. However, increasing the threshold further does not lead to significant improvements in identifying true positives.

Assumption of Log. Reg. (Multicollinearity Check)

```
from statsmodels.stats.outliers_influence import
variance_inflation_factor
import pandas as pd
```

```
import numpy as np

# Function to calculate VIF
def calc_vif(X):
    # Ensure all columns are numeric
    X_numeric = X.select_dtypes(include=[np.number])

    # Check for missing values and handle them
    if X_numeric.isnull().any().any():
        X_numeric = X_numeric.fillna(X_numeric.mean())  # Filling
missing values with the mean

    # Calculate VIF
    vif = pd.DataFrame()
    vif['Feature'] = X_numeric.columns
    vif['VIF'] = [variance_inflation_factor(X_numeric.values, i) for i
in range(X_numeric.shape[1])]
    vif['VIF'] = round(vif['VIF'], 2)
    vif = vif.sort_values(by='VIF', ascending=False)
    return vif

# Calculate VIF after column removal
calc_vif(X)[:5]
```


```
{"summary":"{\n  \"name\": \"calc_vif(X)[:5]\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"Feature\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"installment\",\n          \"open_acc\",\n          \"term\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"VIF\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 45.556908696705925,\
n        \"min\": 13.48,\n        \"max\": 111.2,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          101.49,\n
13.48,\n          43.67\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```


```
# Drop features with high VIF
high_vif_features = ['total_acc', 'annual_inc', 'dti', 'revol_util',
'installment']
X.drop(columns=high_vif_features, axis=1, inplace=True)

# Recalculate VIF after dropping features
calc_vif(X)[:5]
```


```
{"summary":"{\n  \"name\": \"calc_vif(X)[:5]\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"Feature\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
```

```
\"int_rate\",\n          \"pub_rec\",\n          \"open_acc\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"VIF\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 6.191922157133438,\n
\"min\": 4.84,\n        \"max\": 19.48,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          12.88,\n
4.84,\n          6.44\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Validation using KFold

```
# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Set up cross-validation
kfold = KFold(n_splits=5, shuffle=True, random_state=42)  # Adding
shuffle and random_state for reproducibility

# Initialize the Logistic Regression model
logreg = LogisticRegression(max_iter=1000)  # Increase max_iter for
convergence if needed

# Perform cross-validation and calculate the mean accuracy
accuracy = cross_val_score(logreg, X_scaled, y, cv=kfold,
scoring='accuracy', n_jobs=-1)  # Using cross_val_score directly

# Print the result
print(f"Cross Validation accuracy: {accuracy.mean():.3f} ±
{accuracy.std():.3f}")  # Display the mean accuracy with standard
deviation
```

```
Cross Validation accuracy: 0.891 ± 0.001
```

Optimized Insight Statement:

"Both the cross-validation accuracy and testing accuracy are nearly identical, indicating that the model generalizes well to unseen data and is not overfitting or underfitting. This suggests the model is performing reliably and can be trusted for real-world predictions."

This version is clearer and emphasizes the model's reliability and generalization capability, which is crucial for evaluating its performance.

Oversampling using SMOTE

```
sm=SMOTE(random_state=42)
X_train_res,y_train_res=sm.fit_resample(X_train,y_train.ravel())
```

```
<ipython-input-122-2c58a186fd7d>:2: FutureWarning: Series.ravel is
deprecated. The underlying array is already 1D, so ravel is not
```

```
necessary.  Use `to_numpy()` for conversion to a numpy array instead.
  X_train_res,y_train_res=sm.fit_resample(X_train,y_train.ravel())

# Output the shape of the training set after oversampling
print(f"After oversampling, the shape of train_X:
{X_train_res.shape}")
print(f"After oversampling, the shape of train_y: {y_train_res.shape}\
n")

# Output the counts of each class label in the oversampled dataset
print(f"After oversampling, count of label '1' (positive class):
{sum(y_train_res == 1)}")
print(f"After oversampling, count of label '0' (negative class):
{sum(y_train_res == 0)}")

# Initialize and train the logistic regression model
lr1 = LogisticRegression(max_iter=1000)
lr1.fit(X_train_res, y_train_res)

# Predict the labels on the test set
predictions = lr1.predict(X_test)

# Output the classification report
print("Classification Report after Oversampling:\n")
print(classification_report(y_test, predictions))
```

```
After oversampling, the shape of train_X: (396266, 51)
After oversampling, the shape of train_y: (396266,)

After oversampling, count of label '1' (positive class): 198133
After oversampling, count of label '0' (negative class): 198133
Classification Report after Oversampling:

              precision    recall  f1-score   support

           0       0.95      0.80      0.87     84915
           1       0.49      0.81      0.61     20193

    accuracy                           0.80    105108
   macro avg       0.72      0.80      0.74    105108
weighted avg       0.86      0.80      0.82    105108
```

Insights

Impact of Oversampling: After applying oversampling to balance the dataset, the precision and recall scores for both classes (positive and negative) remain similar to those observed in the imbalanced dataset. However, the overall accuracy has decreased, likely due to the model's struggle to accurately predict the positive class (charged off loans).

Performance in Class-Specific Metrics: While the recall for the positive class (charged off) is reasonably high (0.81), the precision (0.49) remains low, indicating that the model tends to misclassify many actual positive class instances as negative. This imbalance in precision and recall points to areas where the model could be further improved.

Room for Improvement: There is still considerable room for improvement, especially for the positive class. Adjustments such as model tuning, feature engineering, or trying more advanced classification techniques could help enhance performance. Additionally, exploring threshold tuning or resampling techniques beyond simple oversampling (like SMOTE) could provide better results for the minority class.

Tradeoff Questions

How can we ensure that our model detects real defaulters while minimizing false positives?

Answer: Given the class imbalance, balancing the dataset can help reduce false positives. For evaluation, we should focus on the macro average F1-score, as it ensures we strike a balance between precision and recall. This is crucial because we don't want to falsely label paying customers as defaulters (false positives), while also making sure we correctly identify defaulters (true positives). Since Non-Performing Assets (NPAs) are a significant concern in this industry, it's essential to be cautious and avoid granting loans to high-risk individuals.

Which features are most important for predicting loan repayment behavior?

Answer: The most important features influencing loan repayment include variables such as annual income, debt-to-income ratio (DTI), and credit history. These factors can help managers identify customers who are more likely to repay their loans fully, ensuring that they focus on high-potential, low-risk borrowers.

Actionable Insights and Recommendations

Loan Payment Distribution:

80% of customers have fully repaid their loans, while 20% are defaulters. The trained model can be utilized to predict whether an individual is likely to repay the loan or default, providing valuable insights for decision-making.

Model Performance:

The model achieves an impressive 94% F1-score for the negative class (Fully Paid), indicating strong performance in predicting loan repayments.

The model has a 62% F1-score for the positive class (Charged Off), indicating room for improvement in identifying defaulters.

Model Reliability:

The model’s cross-validation accuracy and testing accuracy are nearly identical, suggesting reliable generalization to unseen data. This consistency indicates that the model is robust and trustworthy.

Opportunities for Improvement:

While the current performance is solid, further improvement can be achieved by:

Collecting additional data.

Experimenting with more complex models.

Fine-tuning hyperparameters to enhance performance.

ROC AUC Curve:

With an AUC of 0.73, the model is correctly classifying 73% of instances. This is a respectable performance, but there is still potential for optimization.

Precision-Recall Trade-Off:

The precision-recall curve reveals the balance between precision and recall at various thresholds. A higher threshold increases precision but may decrease recall, and vice versa. The ideal threshold should align with business needs and objectives.

Impact of Data Balancing:

After balancing the dataset, a noticeable improvement in both precision and recall scores for both classes was observed, enhancing the model's predictive capabilities.

Test Set Accuracy:

The accuracy of the Logistic Regression Classifier on the test set is 0.892, indicating that the model’s performance is both strong and not a result of mere chance.